

EASTERN INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTING AND INFORMATION TECHNOLOGY



TRƯỜNG ĐẠI HỌC
QUỐC TẾ
MIỀN ĐÔNG
EASTERN
INTERNATIONAL
UNIVERSITY

CAPSTONE PROJECT 1
OFFICE HOURS

Tran Binh Trong – 2231200121

Chau Nhat Tang – 2231200117

Nguyen Dinh Thien – 2231200118

Thai Phan Thanh Sang – 2231200114

Software Engineering

Ho Chi Minh City, August, 2026

**EASTERN INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTING AND INFORMATION TECHNOLOGY**



**TRƯỜNG ĐẠI HỌC
QUỐC TẾ
MIỀN ĐÔNG**

**EASTERN
INTERNATIONAL
UNIVERSITY**

**CAPSTONE PROJECT 1
OFFICE HOURS**

Tran Binh Trong – 2231200121

Chau Nhat Tang – 2231200117

Nguyen Dinh Thien – 2231200118

Thai Phan Thanh Sang – 2231200114

Software Engineering

Ho Chi Minh City, August, 2026

ABSTRACT

The OfficeHours platform has been developed in order to address various issues related to booking and organizing consultations with academic advisors in schools and universities. A large number of institutions face the problems of the manual scheduling, which leads to time conflicts that they are not aware of; therefore, they are unable to run the processes properly. The aim of OfficeHours is to consolidate scheduling processes into one platform and use the available technologies for conflict detection and correct resource distribution.

The solution includes features enabling conflict avoidance in advance, optimal management of waiting lists, user-friendly policy creation, and monitoring the entire scheduling process. No matter if the slots for consultations are available, fully booked, or very popular; it allows knowing the current state of the advisors, as well as the queues of students waiting for an appointment. The students send their requests using the OfficeHours platform and remain aware of what is going on with the scheduling.

The OfficeHours solution is developed based on modern technologies. The web version is created in Next.js and React and provides a user-friendly interface. An important point is that the code base utilizes TypeScript making it easier to work with the system. In addition, the backend technology is based on PostgreSQL, providing both efficient storing of the information and effective exclusion high traffic.

A software solution allows to solve scheduling conflicts, guarantees the efficiency of operations, and maximizes access equity and equality by implementing hybrid allocation schemes that lower the Gini coefficient from 0.25 to 0.10. Future development prospects include such features as campus LMS integration, mobile notifications, and analytical capabilities for forecasting demand based on AI algorithms. The OfficeHours platform is user-friendly and flexible as it meets the requirements of colleges and universities and brings benefits in terms of operational efficiency and access equality.

ACKNOWLEDGEMENT

We would like to sincerely express our appreciation for the help of all who contributed to the accomplishment of the OfficeHours project: their advice and expertise helped a lot.

First of all, we would like to thank the School of Computing and Information Technology (Eastern International University), whose dedication to creating an academic environment in the area of software engineering, distributed systems and algorithms was essential for the implementation of our project.

Next, our thanks go to our supervisor, Mr. Nguyen Tien Tuan Khiem, who provided us with thorough professional advice and guidance and whose remarks made it possible for us to come up with solutions to some technical problems and to improve the quality of our work.

Finally, we cannot forget to thank our project advisors and our group members for their support.

TABLE OF CONTENTS

ABSTRACT	i
ACKNOWLEDGEMENT	ii
TABLE OF CONTENTS	iii
LIST OF FIGURE	vi
LIST OF TABLES.....	viii
LIST OF ABBREVIATIONS	ix
CHAPTER 1. OVERVIEW	1
1.1. Introduction	1
1.2. Project Objectives.....	1
1.3. Challenges	2
1.4. Report Structure.....	2
CHAPTER 2. INTRODUCTION TO TECHNOLOGIES	3
2.1. Next.js and React.....	3
2.1.1. Introduction to Next.js and React.....	3
2.1.2. Advantages of Next.js and React.....	3
2.2. Tailwind CSS.....	3
2.2.1. Introduction to Tailwind CSS.....	3
2.2.2. Advantages of Tailwind CSS	3
2.3. Framer Motion and UI Libraries	4
2.3.1. Introduction to Framer Motion and UI Libraries.....	4
2.3.2. Advantages of UI Libraries	4
2.4. PDF.js	4
2.4.1. Introduction to PDF.js	4
2.4.2. Advantages of PDF.js	4
2.5. Java Spring Boot.....	5
2.5.1. Introduction to Java Spring Boot.....	5
2.5.2. Advantages of Java Spring Boot	5
2.6. PostgreSQL.....	5
2.6.1. Introduction to PostgreSQL.....	5
2.6.2. Advantages of PostgreSQL	5
2.7. JSON Web Token.....	5
2.7.1. Introduction to JSON Web Token	5
2.7.2. Advantages of JSON Web Token.....	6

2.8. Git and GitHub	6
2.8.1. Introduction to Git and GitHub	6
2.8.2. Advantages of GitHub	6
2.9. System Architecture and Technology Stack.....	6
2.9.1. Front-end	6
2.9.2. Back-end.....	6
CHAPTER 3. APPLICATION ANALYSIS, DESIGN AND IMPLEMENTATION	8
3.1. System Description.....	8
3.2. Use Case Diagram	8
3.2.1. General Use Case Diagram.....	8
3.2.2. For Administrator	10
3.2.3. For Lecturer	11
3.2.4. For Student	12
3.3. Activity Diagram	13
3.3.1. Login Function	13
3.3.2. Create Function	13
3.3.3. Read Function.....	15
3.3.4. Update Function	16
3.3.5. Delete Function	17
3.3.6. Password Recovery	17
3.4. Database.....	19
3.4.1. Entity-Relationship Diagram.....	19
3.4.2. Database Analysis	19
3.5. Implementation.....	23
3.5.1. Implementation of Scrum	Error! Bookmark not defined.
CHAPTER 4. EXPERIMENT AND DISCUSSION	24
4.1. Installation Environment	24
4.1.1. Hardware Requirements	24
4.1.2. Software Requirements	24
4.2. Results and Discussion	25
4.2.1. Web Interface Implementation	25
4.2.2. Frontend Evaluation and Discussion	36
CHAPTER 5. CONCLUSION AND FUTURE WORKS	38
5.1. Conclusion.....	38
5.2. Future Works	38

5.2.1. Backend API & Persistence Layer Implementation	38
5.2.2. Mobile Application Development	39
5.2.3. Student Information System (SIS) API Integration.....	39
5.2.4. Real-time Communication & Notification Engine.....	39
5.2.5. Predictive Machine Learning for Demand Forecasting.....	39
REFERENCES	40

LIST OF FIGURE

Figure 1. OfficeHours Academic Consultation System Diagram	9
Figure 2. Administrator Subsystem Diagram	10
Figure 3. Lecturer Subsystem Diagram	11
Figure 4. Student Subsystem Diagram	12
Figure 5. Login Function Diagram	13
Figure 6. Create Function Diagram	14
Figure 7. Read Function Diagram	15
Figure 8. Update Function Diagram	16
Figure 9. Delete Function Diagram	17
Figure 10. Password Diagram	18
Figure 11. Entity-Relationship Diagram	19
Figure 12. Trello – Project Management.....	23
Figure 13. Landing Page.....	25
Figure 14. Public Office Hours Listing	26
Figure 15. Glassmorphic Login Screens	26
Figure 16. Glassmorphic Register Screens.....	27
Figure 17. Password Recovery Flow	27
Figure 18. Role-Aware Dashboard Home	28
Figure 19. My Profile & Security	28
Figure 20. Notification Center.....	29
Figure 21. My Bookings & Timeline View.....	29
Figure 22. Lecturer Search & Directory	30
Figure 23. Import AAO PDF Dashboard	30
Figure 24. Time Table View.....	31
Figure 25. Weekly Slot Picker.....	31
Figure 26. Group Booking & Recurring Setup.....	32
Figure 27. Waitlist Management Dashboard.....	32
Figure 28. Availability Rules Manager	33
Figure 29. Slot Waitlist Queue	33
Figure 30. My Teaching Schedule.....	34
Figure 31. Booking Review & Attendance Journal.....	34
Figure 32. User & Semester Management	35
Figure 33. PDF Timetable Importer	35

Figure 34. Allocation Policy Manager & Audit Log.....36

Figure 35. Analytics & Policy Benchmark Runner.....36

LIST OF TABLES

Table 1. User Table	19
Table 2. Role Table	20
Table 3. Semester Table	20
Table 4. Availability Rule Table	20
Table 5. Schedule Entry Table	20
Table 6. Slot Table	21
Table 7. Booking Table	21
Table 8. Booking Participants Table	22
Table 9. Waitlist Entry Table	22
Table 10. Allocation Event Table.....	22
Table 11. Notification Table.....	22

LIST OF ABBREVIATIONS

No.	Term	Meaning
1	ACID	Atomicity, Consistency, Isolation, Durability
2	AI	Artificial Intelligence
3	AMS	Asset Management Software
4	API	Application Programming Interface
5	CPU	Central Processing Unit
6	CSS	Cascading Style Sheets
7	CSV	Comma-Separated Values
8	DTO	Data Transfer Object
9	EIU	Eastern International University
10	FCFS	First-Come, First-Served
11	FIFO	First-In, First-Out
12	FK	Foreign Key
13	GB	Gigabyte
14	GiST / GIST	Generalized Search Tree
15	HD	High Definition
16	HTML	HyperText Markup Language
17	HTTP	Hypertext Transfer Protocol
18	HYBRID	Hybrid Allocation Model
19	IEEE	Institute of Electrical and Electronics Engineers
20	IoC	Inversion of Control
21	ISO	International Organization for Standardization
22	JPA	Java Persistence API
23	JSON / JSONB	JavaScript Object Notation / Binary JSON
24	JWT	JSON Web Token
25	LMS	Learning Management System
26	LTS	Long-Term Support
27	NEED	Priority-by-Need
28	ORM	Object-Relational Mapping
29	PDF	Portable Document Format
30	PK	Primary Key
31	QR	Quick Response
32	RAM	Random Access Memory
33	RBAC	Role-Based Access Control

34	REST / RESTful	Representational State Transfer
35	RSC	React Server Components
36	SEO	Search Engine Optimization
37	SIS	Student Information System
38	SMTP	Simple Mail Transfer Protocol
39	SQL	Structured Query Language
40	SSE	Server-Sent Events
41	SSD	Solid-State Drive
42	SSR	Server-Side Rendering
43	UI	User Interface
44	URI	Uniform Resource Identifier
45	UTC	Coordinated Universal Time

CHAPTER 1. OVERVIEW

1.1. Introduction

Effective coordination of academic consultation hours is crucial for operational efficiency and educational equity in higher education institutions. In order to tackle the ongoing administrative issues in making student advisors' schedules, preventing overlaps in booking appointments and ensuring fair distribution of consultation availability, our team created OfficeHours - a domain-aware platform created for automated detection of conflicts, easy processing of requests and equity in allocation of resources.

The project yields a reliable software system with a focus on usability, performance and data integrity. For the client's side interface, we used Next.js and React for the development of the dashboard. For the server's side, we implemented TypeScript to provide strict type safety across the server and the client. The system uses PostgreSQL with exclusion constraints based on the date range in order to eliminate double-booking at the database level, which guarantees the resilience of the architecture and maintainability of the solution in the long run.

Following the principles of software engineering and domain research, OfficeHours introduces automated queuing and guarantees algorithmic fairness instead of fragmented processes. By eliminating scheduling problems, reducing the time required for handling requests and assuring equal access for students to the advisors, the platform solves the most pressing operational bottlenecks.

1.2. Project Objectives

The goal of the OfficeHours project is to design a user-friendly platform for effective management and accurate distribution of academic consulting hours that could be used either for normal consultation or high-demand courses. The platform is supposed to provide an effortless and fast interface for students and advisors for booking time slots, avoiding conflicts in the timetable, and observing consultation requests. One of the main objectives of OfficeHours is to create a balanced system between efficient calculations and usability. The platform is able to support personal scheduling, efficient navigation, and real-time status updates that allow users to quickly and rightly find out where they stand on the waiting list. Furthermore, high attention is paid to transaction accuracy and data integrity with various database mechanisms, including the implementation of PostgreSQL restrictions to avoid double-booking, maintain the capacity under peak workload conditions, and protect institutional information. All these characteristics contribute to the creation of trust from users and the improvement of the academic consultation processes. The OfficeHours project is constructed with flexibility for scientific research and future growth of the organization in mind.

The modular architecture implies using a modular allocation engine that can implement and test different policies of fairness: FCFS, Priority-based Need, and Hybrid. The flexibility offered by this platform allows it to develop equity metrics, including a Gini coefficient and wait-time variance, and meet the institutional requirements over time. Overall, the project has been designed to accrue benefits to educational institutions in an efficient distribution of their consulting resources, elimination of structural discrepancies, and provision of data-driven decisions-making.

1.3. Challenges

The development process of OfficeHours involved a number of technological and architectural problems that needed careful consideration and fast solutions. One of the major challenges was creating decisions that would allow engineers to introduce modern web technologies and asynchronous communications into the project. For instance, the usage of Next.js for the creation of responsive front-end and the support of SSE and the implementation of parsing of PDFs on the client-side required careful planning of the architecture to guarantee the proper transfer of data between the elements of the system. Besides, the problem of concurrency arose as well, and specific cases of double bookings during peak periods had to be thoroughly researched and eliminated through the use of PostgreSQL GiST exclusion constraints.

1.4. Report Structure

This report comprises multiple sections that discuss important elements of the OfficeHours project. The report begins with the section on Technological Stack, which explains the decisions concerning the choice of technologies such as Next.js, React, TypeScript, and PostgreSQL. Afterward, the next section on Application Analysis describes the objectives of the project, as well as the components of the data architecture of the application, functional requirements, and database scheme. The third section of the report, devoted to Design and Implementation provides information about the system architecture, UI design, and scheduling mechanisms of the system. The fourth section named Experiment and Discussion describes evaluation experiments performed for the slot allocation policy and presents the results of experiments aimed at measuring system efficiency using the Gini coefficient. Finally, the report concludes with the chapter dedicated to Conclusion and Future Work in which the results of the project implementation are discussed along with possible ways for the improvement of the system in the future.

CHAPTER 2. INTRODUCTION TO TECHNOLOGIES

2.1. Next.js and React

2.1.1. Introduction to Next.js and React

React and Next.js serve as the primary front-end technologies for the OfficeHours platform. These technologies have been used in the implementation of React 19 using Next.js 16 which uses the App Router where Route Groups separate the public non-authenticated routes from authenticated routes like in the case of dashboard routes. Server Components are utilized for both the metadata and Suspense boundaries in which Server Components act as containers that are responsible for component rendering on the client-side. The dashboard is 100% client-rendered, meaning there is no back end at this point. In other words, the auth-gating process happens on the client-side (dashboard)/layout.tsx through the help of the loading skeleton, which does not use server middleware, which could be addressed with an implementation of a backend using Spring Boot.

2.1.2. Advantages of Next.js and React

- Server-Side Rendering and Performance: Generates pre-rendered pages and enhances script processing to improve load speed and performance.
- App Router Architecture: Simplifies page routing and layout control through the use of nested folders.
- Purity and Management: Creates the state during the rendering process to avoid cascades and hydration errors.
- SEO and Accessibility: Makes structured HTML available from the server to ensure visibility to search engines and compliance with the accessibility legislation.

2.2. Tailwind CSS

2.2.1. Introduction to Tailwind CSS

Tailwind CSS uses a utility-focused style system for web applications, utilizing custom semantic design tokens. Instead of random color codes, its design system provides a validated color gradient where it links interactive elements to the color of brand blue (--brand-500) and operational statuses to specific semantic colors (amber for pending, green for confirmed, red for cancelled, and violet for waitlisted).

2.2.2. Advantages of Tailwind CSS

- Standardized Design Architecture: Implements uniform color palettes, font styles, and spacing across all elements.
- Dense Data Presentation: Employs a plain design technique that emphasizes administrative clarity.

- Swift UI Prototyping: Removes custom CSS framework usage by using helper classes directly in components.

2.3. Framer Motion and UI Libraries

2.3.1. Introduction to Framer Motion and UI Libraries

Framer Motion, along with other visual libraries, has made the experience of taking part in an event more memorable. Framer Motion takes care of animation for the transitions and entrances. As for Recharts, it is responsible for all the quantitative data of accessibility indicators. Finally, Rough.js contributes to the interface of landing pages, representing its marketing products differently from management systems.

2.3.2. Advantages of UI Libraries

- Fluid Layout Movement: Framer Motion delivers elegant animated components and stagger effects.
- Accessibility Support: It adheres to `useReducedMotion()` settings so that users bothered by site movement have the heavy animations turned off.
- Accurate Data Graphs: Recharts produces various analytical diagrams of the Gini index based on one color without additional decorations.
- Light Vector Graphics: Lucide offers tree-shakable icons that are capable of changing according to active design tokens.

2.4. PDF.js

2.4.1. Introduction to PDF.js

PDF.js (pdfjs-dist) takes care of the client-side parsing of the students' timetables inside the web client. Being executed inside a non-blocking Web Worker, it parses the students' schedules in the web browser by employing spatial clustering algorithms, which allow for the transformation of raw text blocks into structured JSON time blocks without transferring the computational load to the backend servers.

2.4.2. Advantages of PDF.js

- Client-Side Processes: This method takes away the need for servers to handle the technical load by obtaining the PDF timetable data locally.
- Privacy-Protecting: This process ensures that important documents containing sensitive information about students are processed only within the client browser.
- Deterministic Translation: Transforms the text coordinates in space into structured representations of the timetable with precision and accuracy.

2.5. Java Spring Boot

2.5.1. Introduction to Java Spring Boot

The backend setup is built as a performant modular monolith based on Java and Spring Boot for the purposes of designing a RESTful Web API which unifies schedule management, conflict resolution, and allocation of time slots through an algorithm. The solution utilizes Spring Data JPA to support relational data and Jakarta validation to validate the requests.

2.5.2. Advantages of Java Spring Boot

- Enterprise Architecture: Features solid modular technology that offers IoC and dependency injection for long-term use.
- Strong Security Framework: Can be used with Spring Security for thorough authorization and role-based access control.
- ORM and Transaction Management: Implements easy access to data via Spring Data JPA/Hibernate while maintaining transaction security.
- High Performance and Scalability: Effectively manages numerous requests and complicated business process execution.

2.6. PostgreSQL

2.6.1. Introduction to PostgreSQL

PostgreSQL 15+ is used as the principal relational database, ensuring adherence to ACID standards and stringent constraint application. It uses a number of extensions, such as btree_gist and citext, which help in the application of exclusion constraints on a database level that operate mathematically to avoid double-booking and enforce case-insensitive email uniqueness.

2.6.2. Advantages of PostgreSQL

- The integrity of ACID: Provides assurance of the complete accuracy of transactions when handling several requests at the same time.
- Exclusion constraints based on GiST technology: Combines different types of logic with time ranges to prevent making double bookings.
- The precision of time zones: Provides protection from inaccuracy in time measurement across different regions for clients through the support of the native type of data.

2.7. JSON Web Token

2.7.1. Introduction to JSON Web Token

JSON Web Tokens are used for user authentication without having sessions, as well as for implementing role-based access control (RBAC). Passwords are encrypted using an irreversible algorithm called bcrypt before they are stored. Tokens include an identity payload that is signed by a cryptographic method and dictates access rules for end-users falling into one of three roles

- STUDENT, LECTURER, and ADMIN, while also implementing a concept of data sovereignty on the record level.

2.7.2. Advantages of JSON Web Token

- Authentication without state information: This allows for faster processing of API requests since there is no server-side session lookup required.
- Specific/access-focused access control: The authorization of endpoints and enforcement of row-level security is carried out as required by the user.
- Security through encryption: Security against tokens being tampered with.

2.8. Git and GitHub

2.8.1. Introduction to Git and GitHub

GitHub and Git offer the tools and services necessary for managing code, tracking its versions, and collaborating on software development projects. The database models are updated through a series of migration scripts that are maintained in GitHub, allowing us to keep development, staging, and production versions synchronized.

2.8.2. Advantages of GitHub

- Version Control and Collaboration: Keeps records of code updates and manages the processes from various developers.
- Relational Schema Migration: Provides the same type of schema in various deployment environments through database migration versions.

2.9. System Architecture and Technology Stack

2.9.1. Front-end

- Next.js & React: The basic framework is used for server-side rendering, routing, and component creation.
- Tailwind CSS: Utility-first framework of styles with the unique token setup.
- Framer Motion: Animation editor with integrated accessibility controls.
- Recharts & Lucide: Engines used for data analysis of the metrics of equity and collection of vector icons.
- PDF.js: Web Worker-based application for PDF timetable reading.

2.9.2. Back-end

- Java Spring Boot: Modular RESTful backend API handling logic for scheduling, allocation policies and transactions.
- Spring Security & JWT: Stateless authentication engine that provides role-based security and password encryption using the BCrypt algorithm.

- PostgreSQL 15+: This is a relational database that uses exclusion constraints of btree_gist type that prevent conflicts.
- GitHub & Migrations: These are used for maintaining the version control repository for the databases and database migration.

CHAPTER 3. APPLICATION ANALYSIS, DESIGN AND IMPLEMENTATION

3.1. System Description

The OfficeHours Academic Consultation Platform is intended to automate scheduling processes between students and faculty by creating a balanced and effective system for consultations. The program is built on a robust Java Spring Boot RESTful Web API backend enabling Next.js to function. The graphical interface is responsive and based on a CSS framework. The database used is PostgreSQL with exclusion constraints serving the purpose of preventing double-booking, while Redis technology is applied for caching the outcomes of complex calculations and operations.

The platform is designed into three specialized interfaces that correspond to the functions of Students, Lecturers, and Administrators:

- Student Interface: Designed to give students scheduling capabilities and options for monitoring interactions. Students can review lecturers' schedule directories, upload timetable PDFs to avoid overlapping schedules, request consultation appointments, and automatically sign up for pending waitlists. The interface will assist students in tracking their requests, meetings, and the waitlist status.
- The interface for lecturers is focused on managing appointments from the lecturer side. Lecturers can set templates for regular weekly appointments, plan exceptions for certain days, check the incoming requests for appointments, record the attendance of students, and keep track of meetings.
- The Administrator interface is aimed at managing the system globally and allowing administrators to oversee the allocation of appointments. Administrators manage accounts for users and academic semesters, upload timetables in bulk by parsing PDF files, set some rules for waiting list generation, and also monitor the distribution of appointments via analytical dashboards using Gini coefficient indicators.

By introducing real-time communication, workflow processing, and algorithmic fairness engines, OfficeHours makes and speeds up academic counseling processes and creates transparent and fair access to academic expertise.

3.2. Use Case Diagram

3.2.1. General Use Case Diagram

The OfficeHours platform provides basic operational activities divided among various system roles to establish an effective academic consulting process.

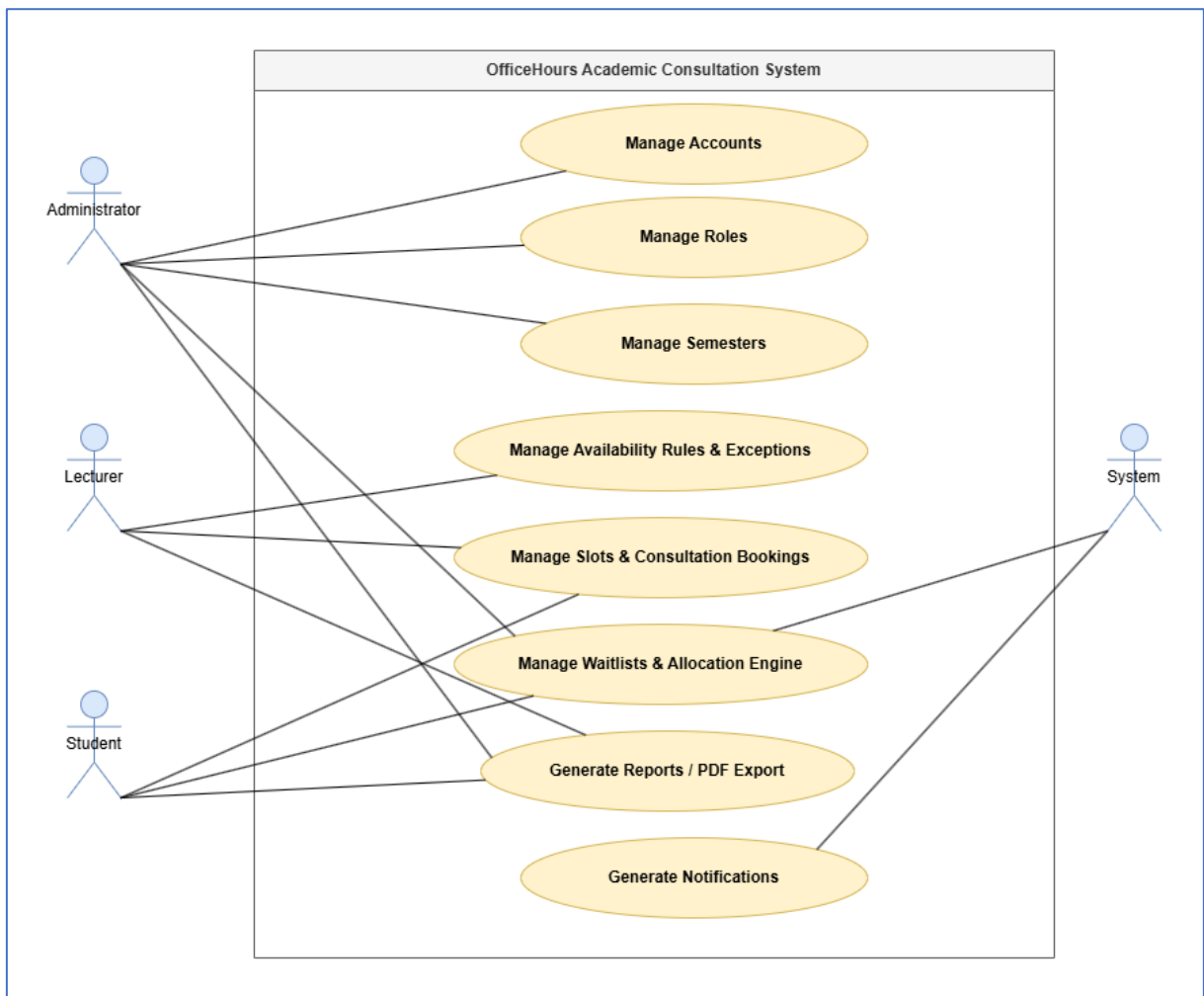


Figure 1. OfficeHours Academic Consultation System Diagram

- Account Management: This function enables Administrators to create, edit, deactivate, and assign roles (STUDENT, LECTURER, ADMIN) to system users to provide operational safety and confidentiality.
- Role Management: This feature allows Administrators to regulate the limits of permissions and access for different user layers.
- Semester Management: This function ensures creating and managing institutional academic terms defining recurring scheduling changes.
- Managing Rules of Availability and Exceptions: Lecturers can define recurring weekly availability for themselves along with exceptions on specific dates by adding additional slots or blocking some time periods when they are unavailable.
- Managing Slots and Consultation Reservations: Available for Students as well as for Lecturers, who can approve or deny any requests after students manage to find and pick slots without conflicts.

- **Manage Waitlists & Allocation Engine:** Allows Students to join waitlists for full slots, while the System and Administrator manage automated algorithmic allocations and manual overrides.
- **Generating Reports and Exporting in PDF:** This functionality allows for obtaining consultation logs, timetable entries, and allocations via PDF or CSV.
- **Preparation of Notifications:** Notifications are produced by the system through SSE and email (SMTP) and come as notices about booking confirmations, cancellations, and pending requests.

3.2.2. For Administrator

The Administrator is responsible for all activities carried out in the system as well as system configuration and user authorization.

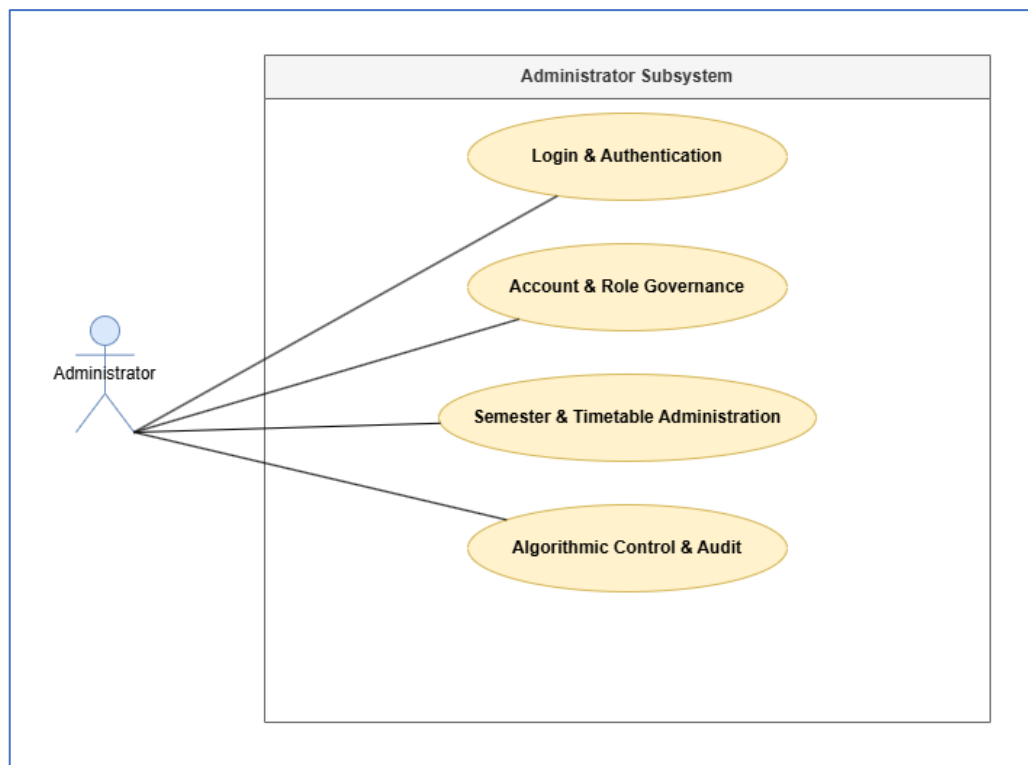


Figure 2. Administrator Subsystem Diagram

- **Authentication and Login:** We back credentials verification and password restoration processes.
- **Governance of Role and Account:** Admin can see the list of all the accounts, create new users, change the role/department of the organization, enable or disable user access.
- **Administration of semester and timetable:** We define the active semester limits, and execute the In-Browser PDF timetable scanner which students and lecturers use on behalf of any user that cannot serve by themselves - it uses the lecturer/student option

to select the account; then, either AAO PDF upload or normal typing allow us to get information directly into their schedule and the separate slot search platform checks available slots through all the lecturers.

- Auditing and Control through the algorithm: It selects current waitlist allocation policies, conducts the manual intervention in case it is needed and provides oversight of the Gini coefficient equity dashboard.

3.2.3. For Lecturer

Lecturers monitor the consultation availability and track the appointment requests and outcomes of offering advice.

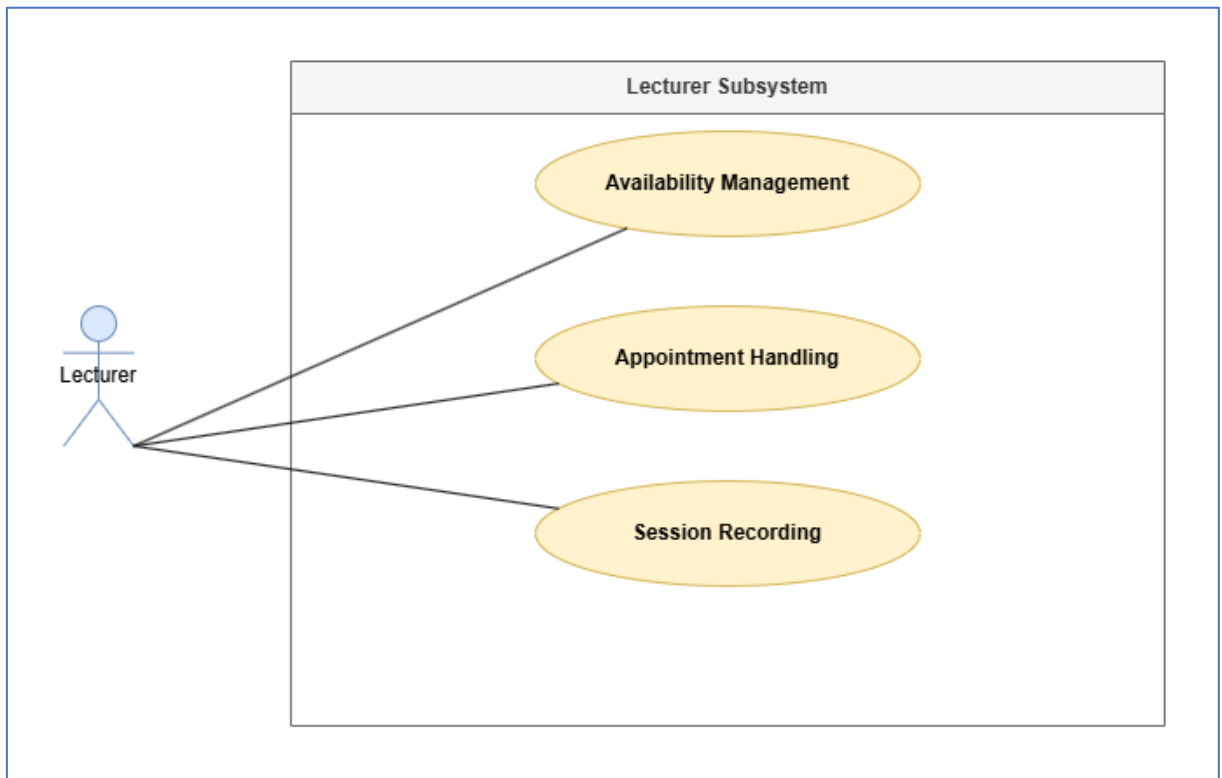


Figure 3. Lecturer Subsystem Diagram

- Availability Management: Lecturers prepare templates with constant meeting times for consultations on a regular basis and indicate changes for a specific day on the agenda.
- Synchronization of schedule: Lecturers make use of their own AAO teaching timetable PDF (which is also used for students) to create their teaching timetable automatically or add/remove class blocks manually.
- Appointment Handling: Lecturers receive notifications about booked consultations from students and have the right to accept or refuse students' requests on any consultation.
- Session Recording: Professors register the attendance statuses (such as Present, Absent, Did not show) and include their mentoring notes.

3.2.4. For Student

Students browse availability, submit consultation requests, and track scheduling progress.

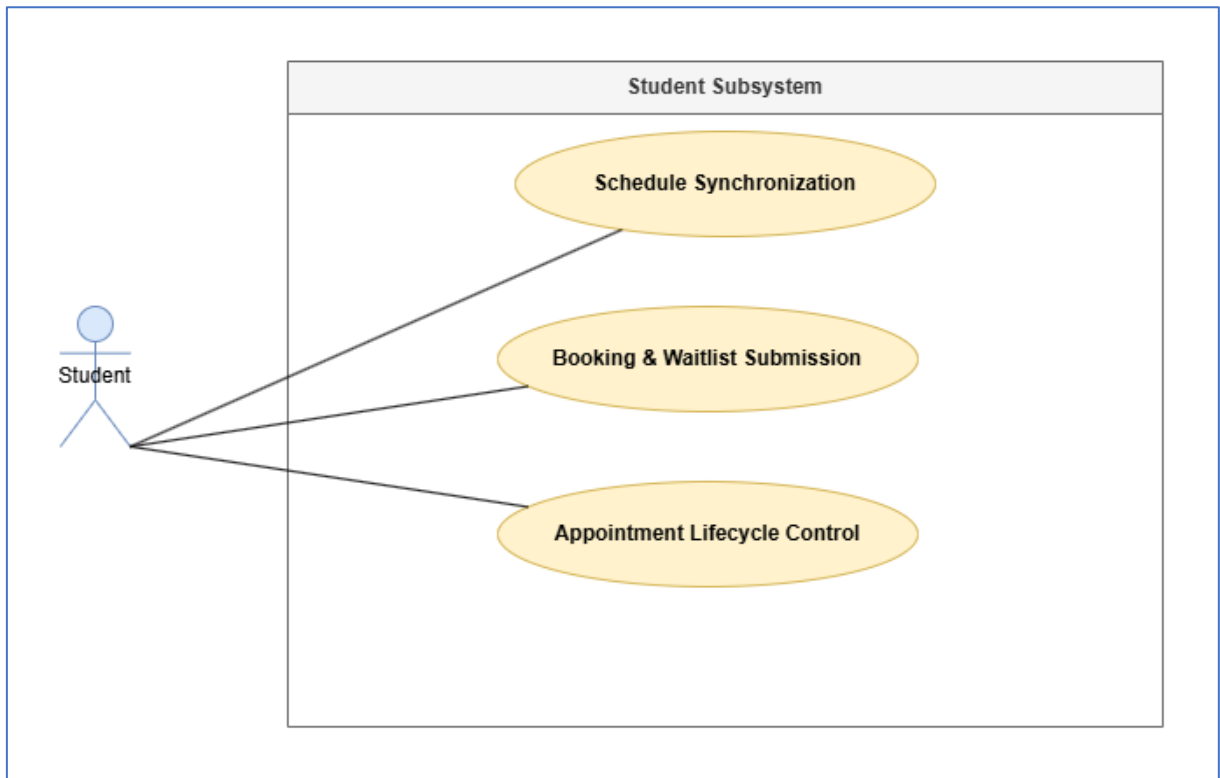


Figure 4. Student Subsystem Diagram

- **Schedule Synchronization:** By scanning their respective AAO timetables, students are able to see their times in two forms: Timetable Matrix and Agenda List.
- **Booking & Waitlist Submission:** Students can book their appointments according to their preference, hold a group appointment with their colleagues or get on the waiting list.
- **Appointment Lifecycle Control:** Students can see whether their request has been accepted or not, and cancel or reschedule their appointments on time.

3.3. Activity Diagram

3.3.1. Login Function

Shows how authentication of users is done using Spring Security and JWT technology.

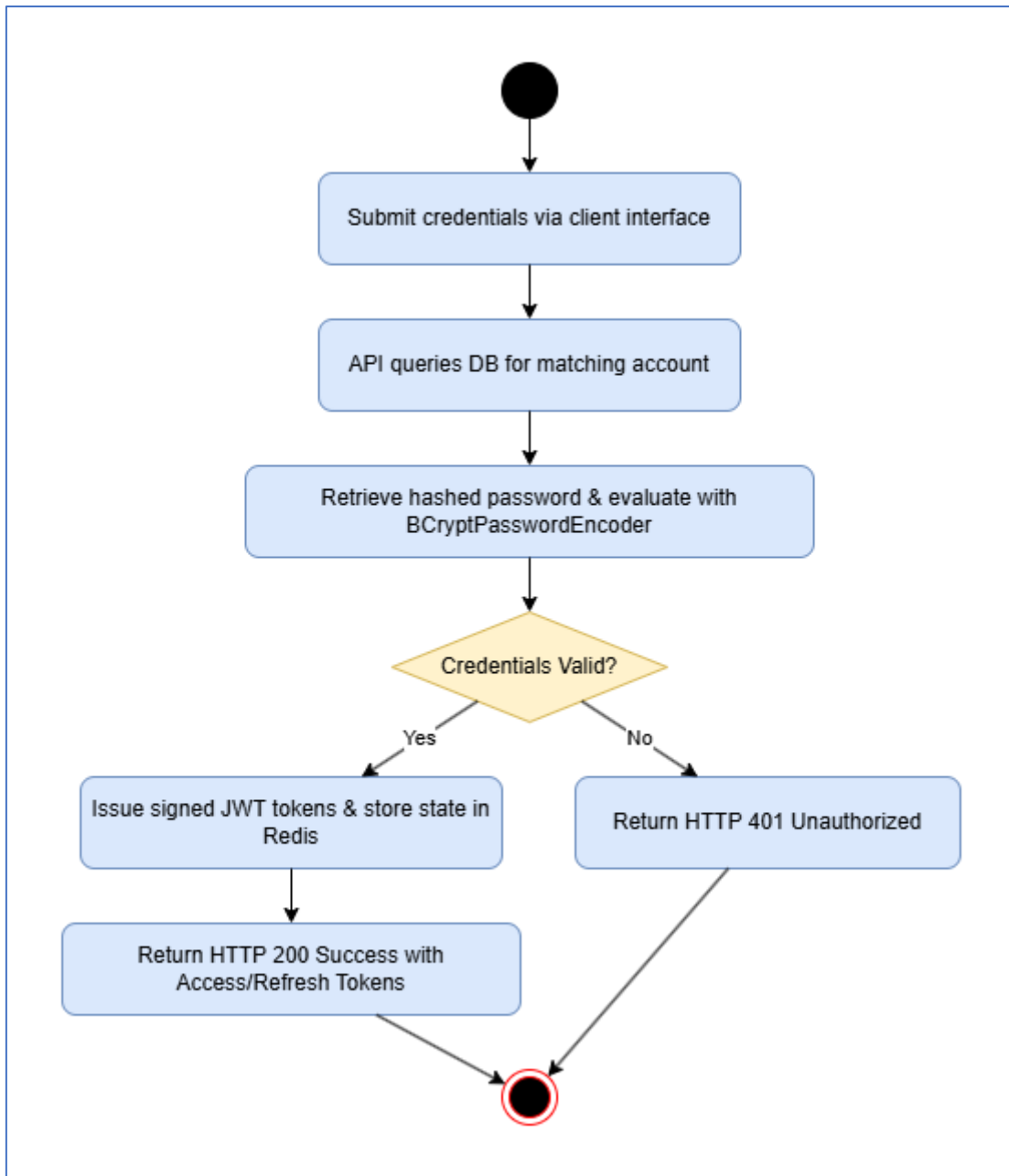


Figure 5. Login Function Diagram

The user enters their credentials in the client lobby. The API gets the record from the database and so retrieves the hashed passwords and checks with the technology known as BCryptPasswordEncoder. When it is checked ok, the server sends JWT tokens.

3.3.2. Create Function

Demonstrates efforts to create new entity records (e.g., availability slots or booking requests) while also validating concurrency at the database level.

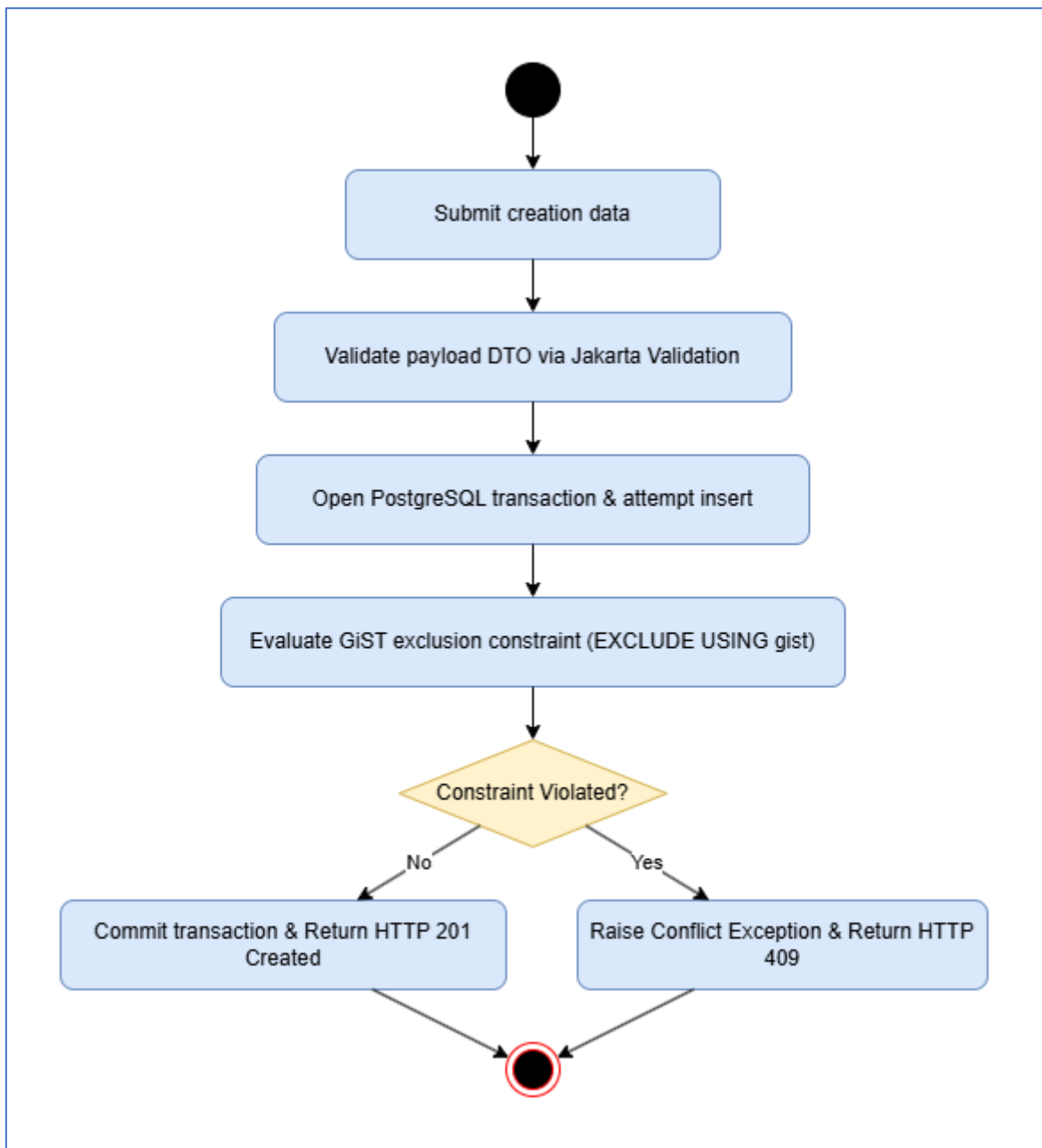


Figure 6. Create Function Diagram

Once the user sends the creation data, the Spring Boot API validates it through Jakarta Validation. A transaction for inserting a record in PostgreSQL will be initiated. For bookings, PostgreSQL will enforce the GiST exclusion constraint (EXCLUDE USING gist). If insertions succeed, HTTP 201 Created will be returned, while if a conflict arises, HTTP 409 is returned.

3.3.3. Read Function

Shows the process of fetching and filtering slot or user data using Redis caching.

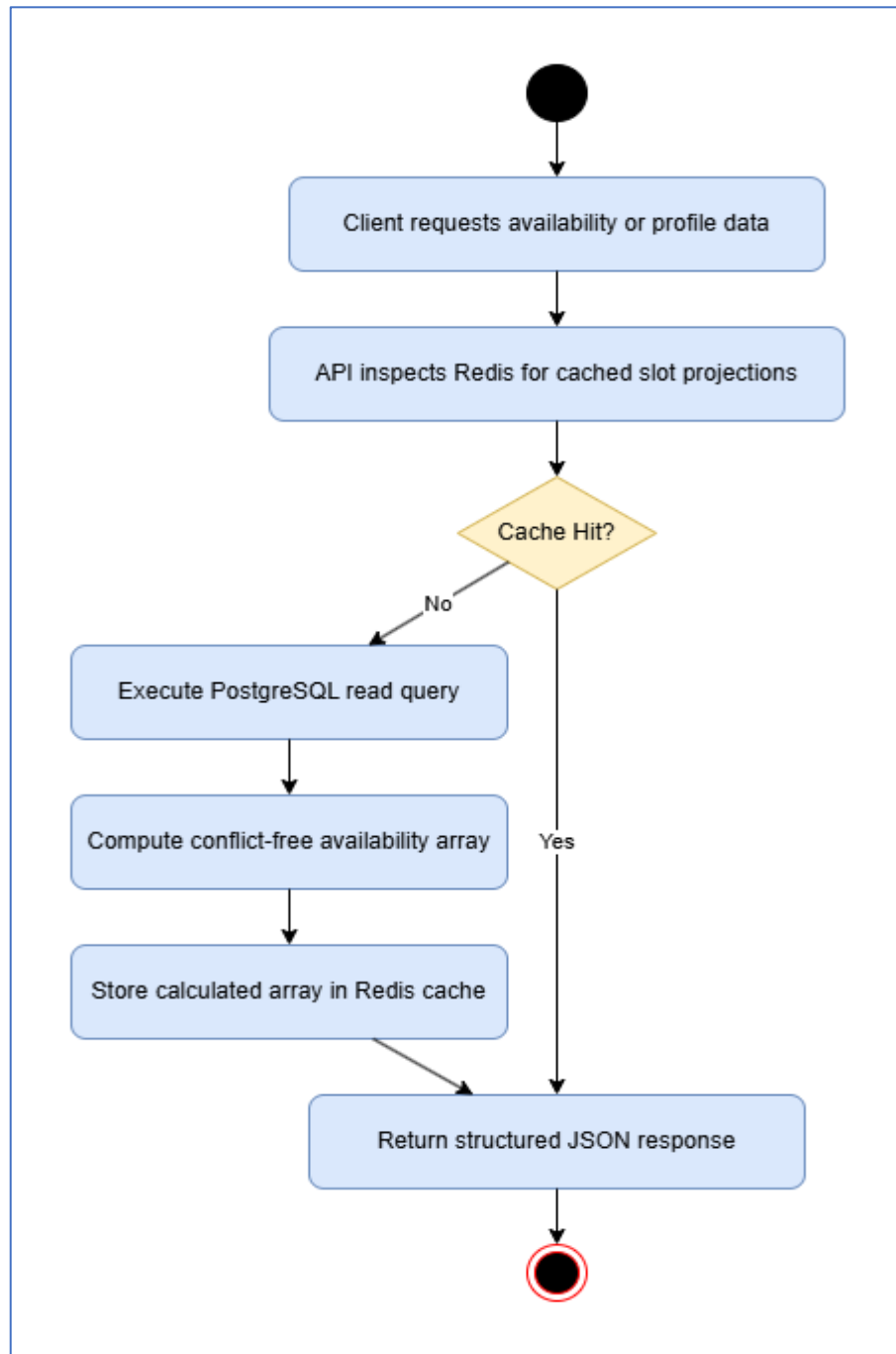


Figure 7. Read Function Diagram

The client issues a request for getting availability or profile data. The API checks Redis cache for cached slot projections. If the cache is successful, the data will be returned immediately; if it fails, a PostgreSQL read query is used to find out availability by subtracting scheduled blocks, and the array will be cached in Redis.

3.3.4. Update Function

Describes the modification of the current entities, such as changing of the availability exceptions or modification of booking statuses.

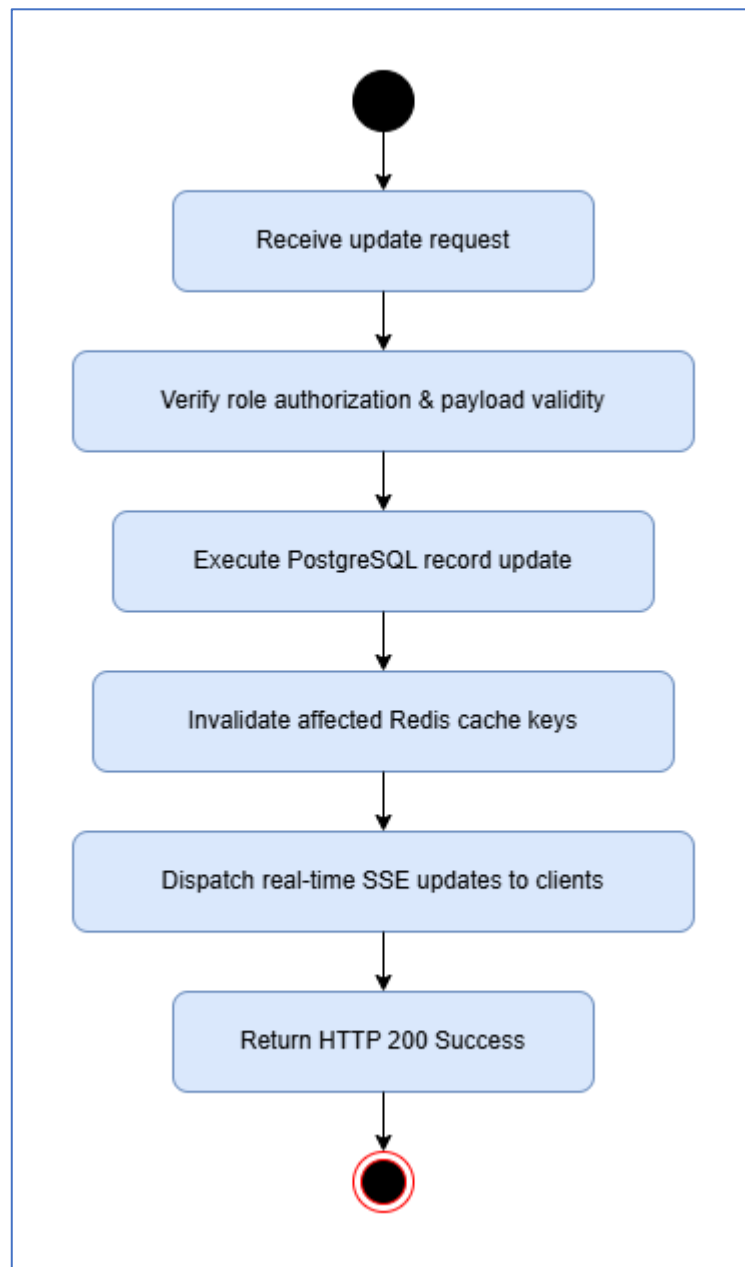


Figure 8. Update Function Diagram

A request for an update is received by the system followed by role verification and payload validation. After PostgreSQL has made changes to the database, Spring Boot invalidates the Redis cache by deleting relevant keys and sending notifications using server sent events.

3.3.5. Delete Function

Describes the rules governing cancellation of bookings and removing of entities.

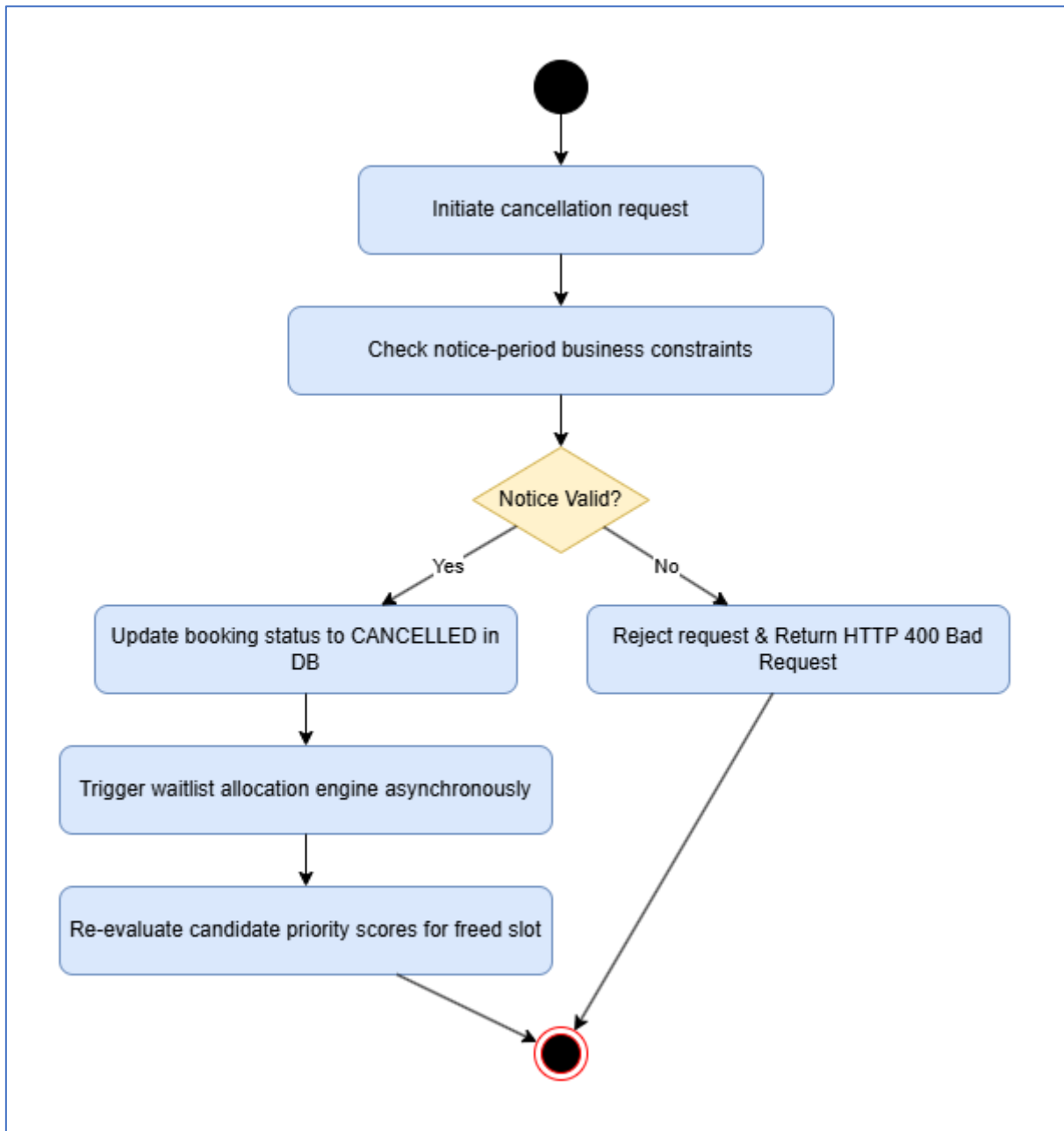


Figure 9. Delete Function Diagram

When the user requests to cancel the booking, the system checks whether the cancellation request satisfies the notice period definitions set up. Upon the successful check, the booking status is set as CANCELLED. This in turn starts the cancellation of waiting list allocation engine which verifies candidate priority scores of the free place.

3.3.6. Password Recovery

Gives an example of a token-based way of recovering the password of an account.

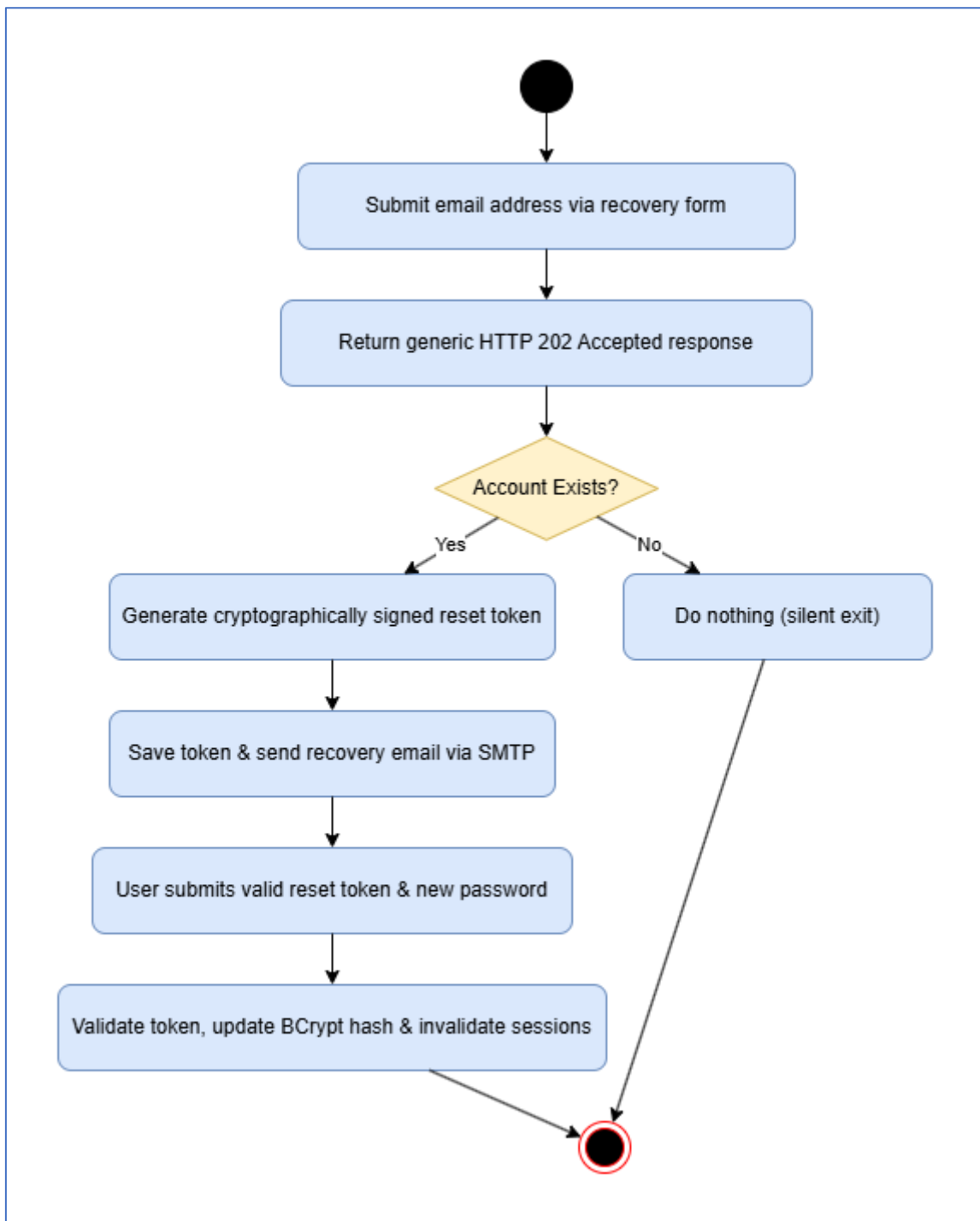


Figure 10. Password Diagram

The email address is entered by the user. The system returns a response of the type HTTP 202 Accepted meaning that it is returned no matter if the account exists. If the account exists, the reset token is signed cryptographically and successfully sent to the user's email as an email message. When the user enters the token, the password can be updated.

3.4. Database

3.4.1. Entity-Relationship Diagram

The design of the relational data storage is based on key entities that manage identity, study timetables, consultation, bookings, and waiting lists.

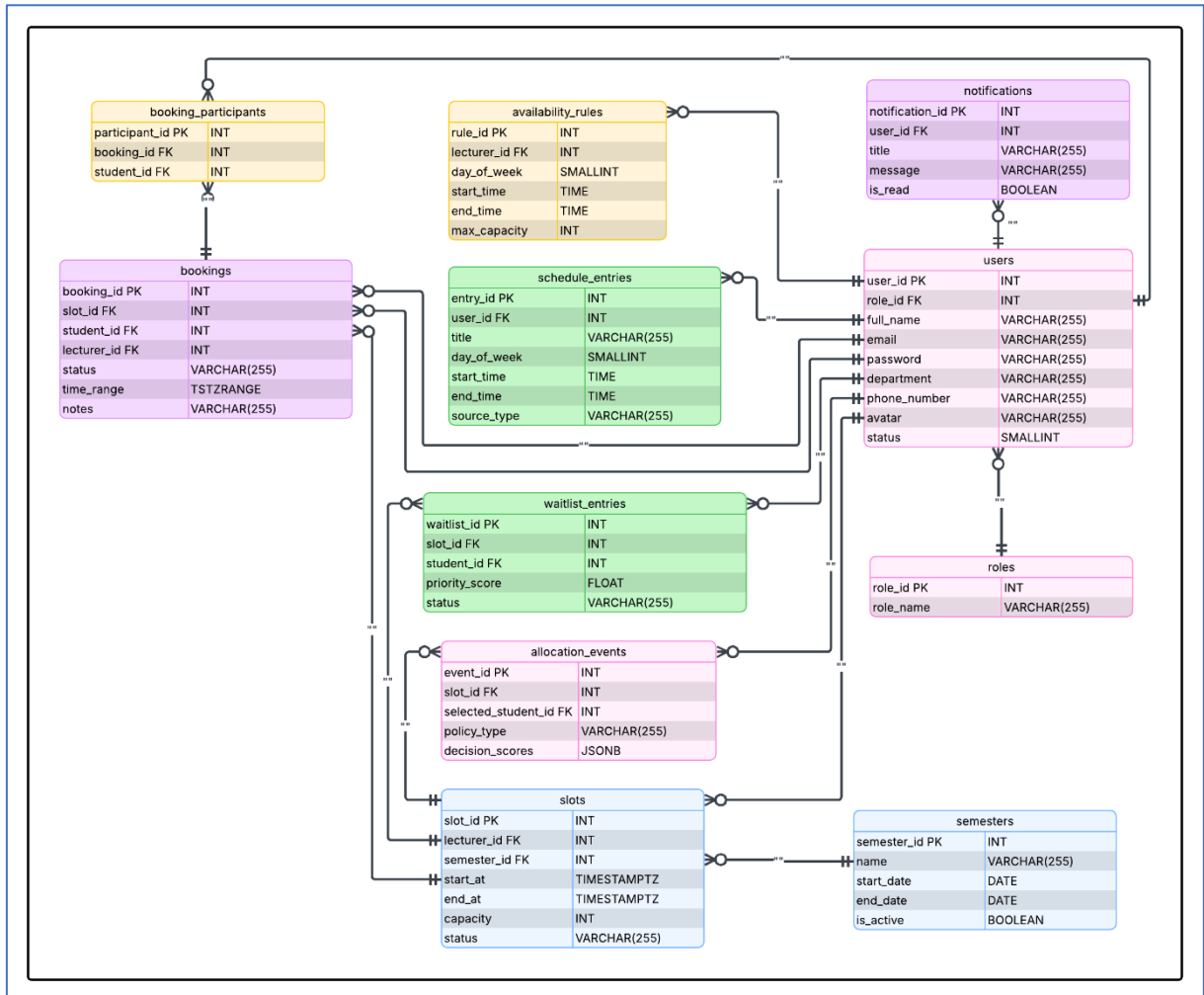


Figure 11. Entity-Relationship Diagram

3.4.2. Database Analysis

Users Table: Contains information regarding users' credentials and roles

Table 1. Users Table

No.	Field Name	Type	Description
1	user_id	int (PK)	Unique primary key identifier
2	role_id	int (FK)	References roles(role_id)
3	full_name	varchar(255)	User full name
4	email	varchar(255)	Unique case-insensitive email address
5	password	varchar(255)	BCrypt encrypted password hash
6	department	varchar(255)	Academic department affiliation

7	phone_number	varchar(255)	Contact telephone number
8	avatar	varchar(255)	Profile image URI
9	status	tinyint	Account status (1 = Active, 0 = Inactive)

Roles Table: Contains information about role assignments.

Table 2. Roles Table

No.	Field Name	Type	Description
1	role_id	int (PK)	Unique role identifier
2	role_name	varchar(255)	Name of role (STUDENT, LECTURER, ADMIN)

Semesters Table: Contains information with respect to which semester it currently is, as well as has details on the past semesters.

Table 3. Semesters Table

No.	Field Name	Type	Description
1	semester_id	int (PK)	Primary key identifier
2	name	varchar(255)	Semester title (e.g., "Fall 2026")
3	start_date	date	Term start boundary
4	end_date	date	Term end boundary
5	is_active	boolean	Flag for the currently active global semester

Availability Rules Table: Keeps templates for weekly consultations shown by lecturers.

Table 4. Availability Rules Table

No.	Field Name	Type	Description
1	rule_id	int (PK)	Unique rule identifier
2	lecturer_id	int (FK)	Foreign key referencing users(user_id)
3	day_of_week	tinyint	Day index (1 = Monday, ..., 7 = Sunday)
4	start_time	time	Daily window start time
5	end_time	time	Daily window end time
6	max_capacity	int	Maximum allowed students per generated slot

Schedule Entries Table: Contains the schedule of busy slots of the students and lecturers.

Table 5. Schedule Entries Table

No.	Field Name	Type	Description
1	entry_id	int (PK)	Primary key identifier

2	user_id	int (FK)	Owner user identifier
3	title	varchar(255)	Course or event title
4	day_of_week	tinyint	Day index
5	start_time	time	Event start time
6	end_time	time	Event end time
7	source_type	varchar(255)	Import origin (PDF_PARSED, CSV_IMPORTED, MANUAL)

Slots Table: Contains data about available consultations for the students.

Table 6. Slots Table

No.	Field Name	Type	Description
1	slot_id	int (PK)	Unique slot identifier
2	lecturer_id	int (FK)	Host lecturer ID
3	semester_id	int (FK)	Related academic semester ID
4	start_at	timestampz	Slot start timestamp in UTC
5	end_at	timestampz	Slot end timestamp in UTC
6	capacity	int	Student capacity limit
7	status	varchar(255)	Current slot state (OPEN, FULL, CLOSED)

Bookings Table: Contains information about students' appointments with consultation slots.

Table 7. Bookings Table

No.	Field Name	Type	Description
1	booking_id	int (PK)	Primary key identifier
2	slot_id	int (FK)	Foreign key to slots(slot_id)
3	student_id	int (FK)	Student user ID
4	lecturer_id	int (FK)	Denormalized lecturer ID for exclusion constraints
5	status	varchar(255)	Booking status (PENDING, CONFIRMED, DECLINED, CANCELLED)
6	time_range	tstzrange	Denormalized PostgreSQL range for GiST index
7	notes	varchar(255)	Post-session notes or student request notes

Booking Participants Table: Allows several students to make booking for one consultation together.

Table 8. Booking Participants Table

No.	Field Name	Type	Description
1	participant_id	int (PK)	Unique record identifier
2	booking_id	int (FK)	Primary booking reference
3	student_id	int (FK)	Additional participating student ID

Waitlist Entries Table: Keeps data about which student is in the line waiting for a free consultation slot.

Table 9. Waitlist Entries Table

No.	Field Name	Type	Description
1	waitlist_id	int (PK)	Primary key identifier
2	slot_id	int (FK)	Target full slot ID
3	student_id	int (FK)	Queued student ID
4	priority_score	float	Computed priority metric for fair allocation
5	status	varchar(255)	Entry status (QUEUED, OFFERED, EXPIRED, ACCEPTED)

Allocation Events Table: Tracks allocation events related to waitlists

Table 10. Allocation Events Table

No.	Field Name	Type	Description
1	event_id	int (PK)	Primary key identifier
2	slot_id	int (FK)	Subject slot ID
3	selected_student_id	int (FK)	Selected recipient student ID
4	policy_type	varchar(255)	Applied policy (GINI_FAIRNESS, FIFO, MANUAL_OVERRIDE)
5	decision_scores	jsonb	Serialized candidate priority scores for research audit

Notifications Table: Contains data on notifications sent to users.

Table 11. Notifications Table

No.	Field Name	Type	Description
1	notification_id	int (PK)	Primary key identifier
2	user_id	int (FK)	Recipient user ID
3	title	varchar(255)	Brief alert headline

4	message	varchar(255)	Main notification payload
5	is_read	boolean	Read/Unread flag

3.5. Implementation of Scrum

Incremental development and team collaboration within the software development process were accomplished through the application of the Scrum agile project management framework by the team. All tasks were performed during iterations of work lasting two weeks and called sprints.

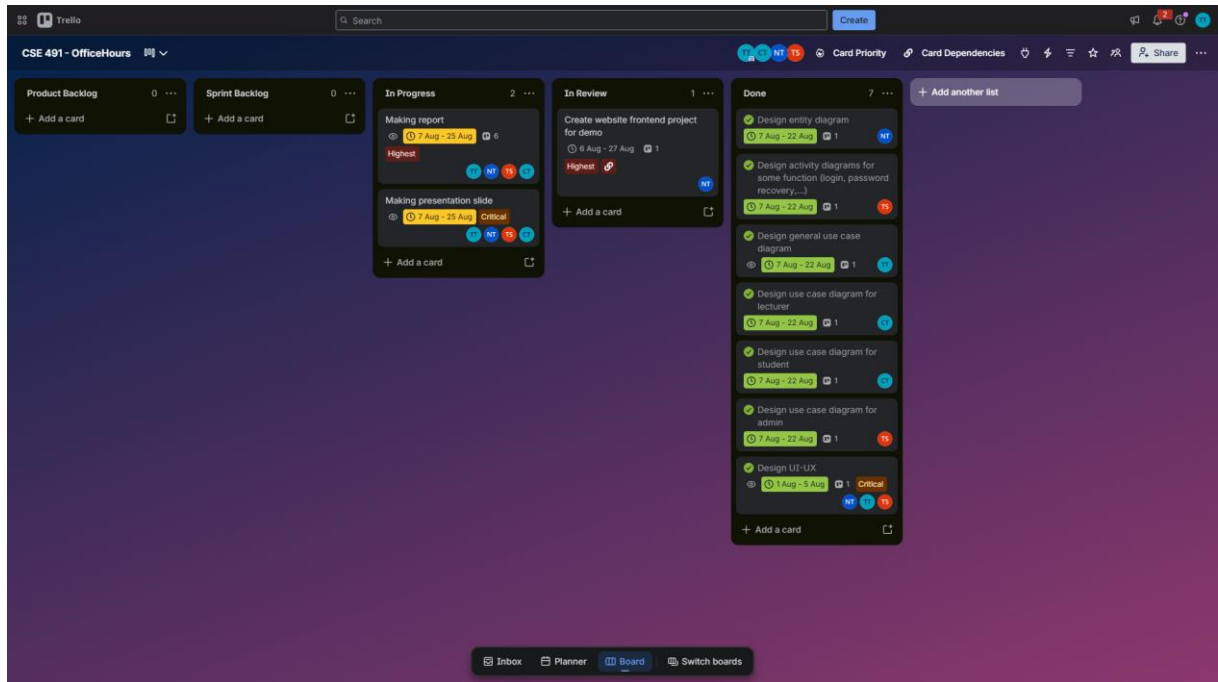


Figure 12. Trello – Project Management

In the course of the sprint planning, the team reviewed the prioritized backlog items and divided them into sprint backlog items. They performed their daily meetings in order to review and assess the progress in their work, help remove any obstacles associated with the technical implementation of the tasks, and synchronize the work of the front-end and back-end teams.

At the end of every sprint, the team performed sprint reviews and evaluate their product increments according to their functionality.

CHAPTER 4. EXPERIMENT AND DISCUSSION

4.1. Installation Environment

4.1.1. Hardware Requirements

The development setup allows for the progress of client-side component production, as well as for hero testing performance and user interface function of the internet. The hardware should meet at least the following requirements:

- Processor: Quad-core CPU (e.g., Intel Core i5 10th Gen, AMD Ryzen 5, or Apple M1/M2) to enable working with hot reloading and site bundling activities.
- Memory (RAM): Minimum 8 GB (preferred 16 GB) for ensuring easy development testing with the developer utility, development server, and clients' browsers.
- Storage: At least 10 GB of free SSD space required for node modules' dependencies, website assets, developer utilities, and results of the build process.
- Display: 1080p Full HD (1920x1080), so that it is possible to assess the performance of the website.
- Network: Possibility to rely on a stable internet connection for downloading npm modules, as needed.

4.1.2. Software Requirements

To achieve effective development as well as deployment of the web interface using front-end technologies, several software applications will be used, comprising:

- Operating System: Windows 10/11, macOS (most likely macOS Monterey 12.x and above), Linux (for example, Ubuntu 22.04 LTS).
- Run-time and package manager: Node.js (v18.x or v20.x) and npm/pnpm (which is used for executing browser scripts and maintaining JavaScript and TypeScript packages).
- IDE: Visual Studio Code with ESLint, Prettier and Tailwind CSS plugins installed.
- Browsers are the newest, most advanced versions of the three most popular internet browsers – Google Chrome, Mozilla Firefox and Microsoft Edge. The said browsers will be employed for the purpose of testing the layout of the webpages and for viewing the graphics on them.
- Front-end libraries and frameworks:
 - + Next.js will be used for design of UI based on components.
 - + Tailwind CSS is targeted for utility-first responsive design.
 - + Framer Motion for transitions of UI states.
 - + Recharts for displaying analytics.

- + PDF.js is aimed at reading the schedule on client-side.

4.2. Results and Discussion

4.2.1. Web Interface Implementation

Public & Authentication Views:

- Landing Page: This distinctive visually appealing page contains a "Neo-Brutalist" design that utilizes sharp and dominating colors to deliver rough looks using a method called rough.js.

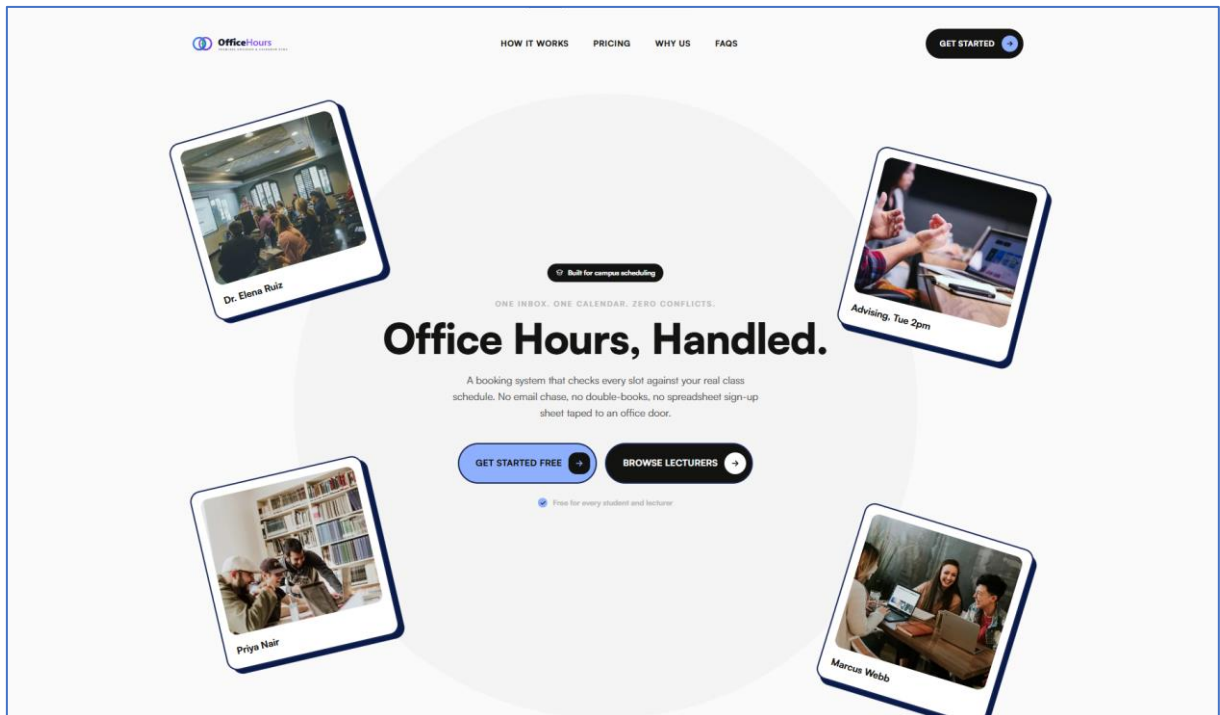


Figure 13. Landing Page

- Public Office Hours Listing: This listing contains data on public office hours in various departments without any need for authentication.

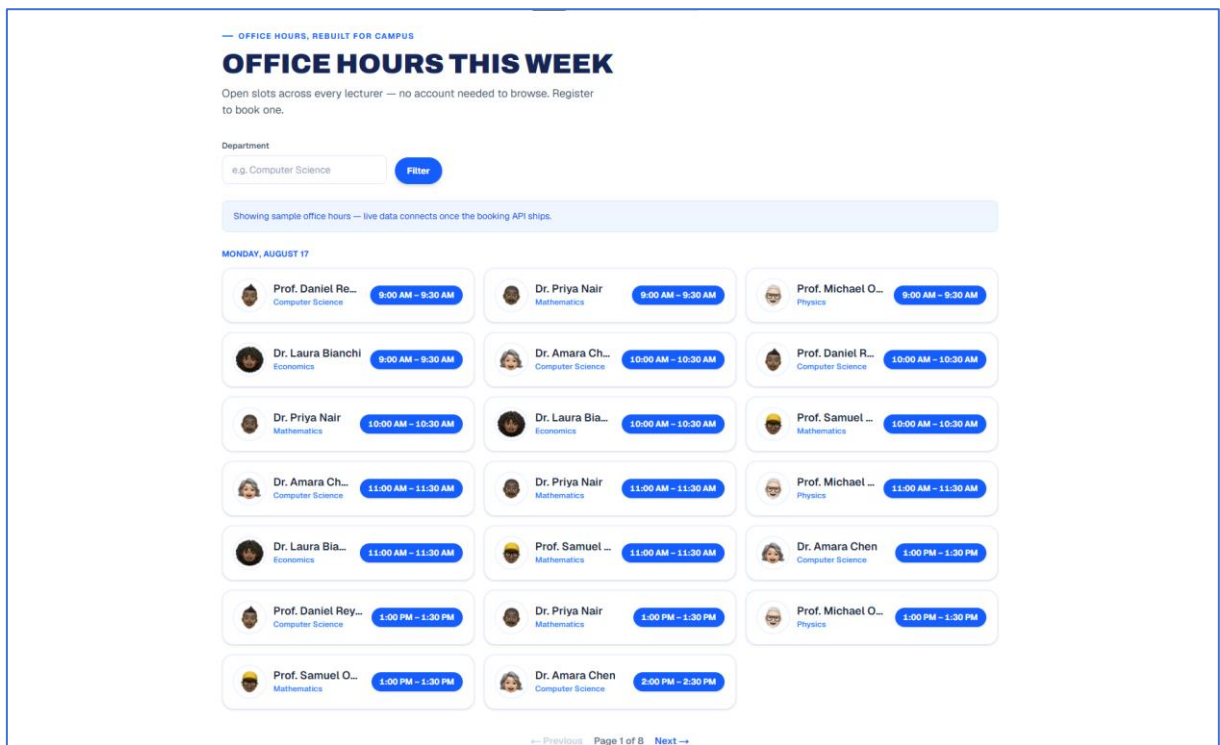


Figure 14. Public Office Hours Listing

- Glassmorphic Login & Register Screens: The screen design features CSS filter with a "backdrop: blur(20px)" that impressively compliments the design with semi-transparent containers placed over a video display.

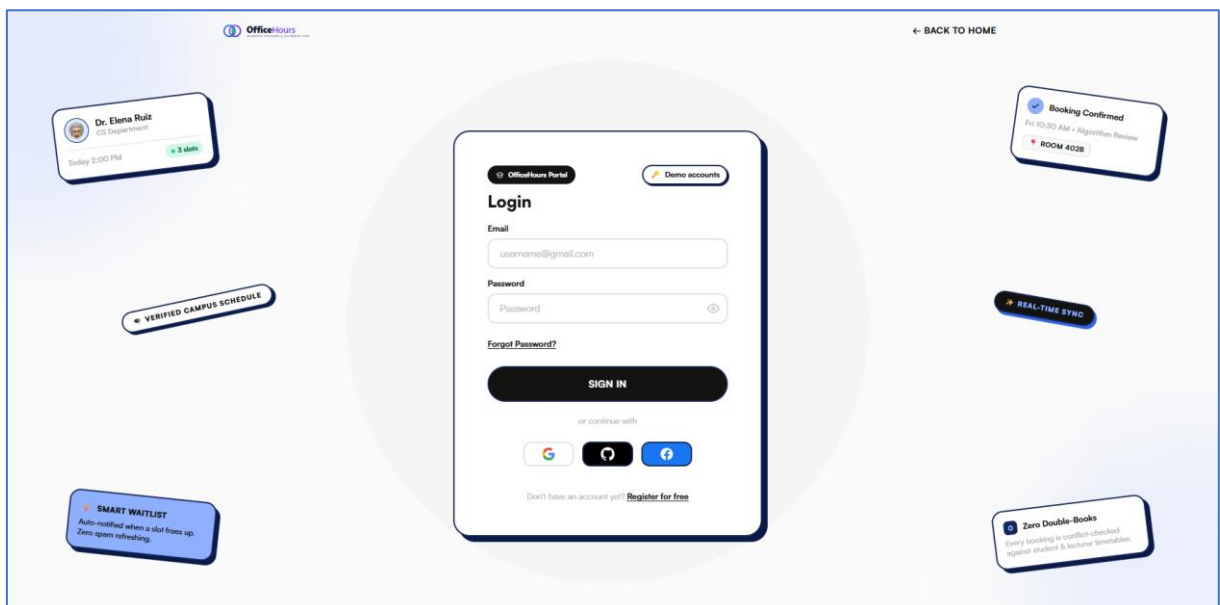


Figure 15. Glassmorphic Login Screens

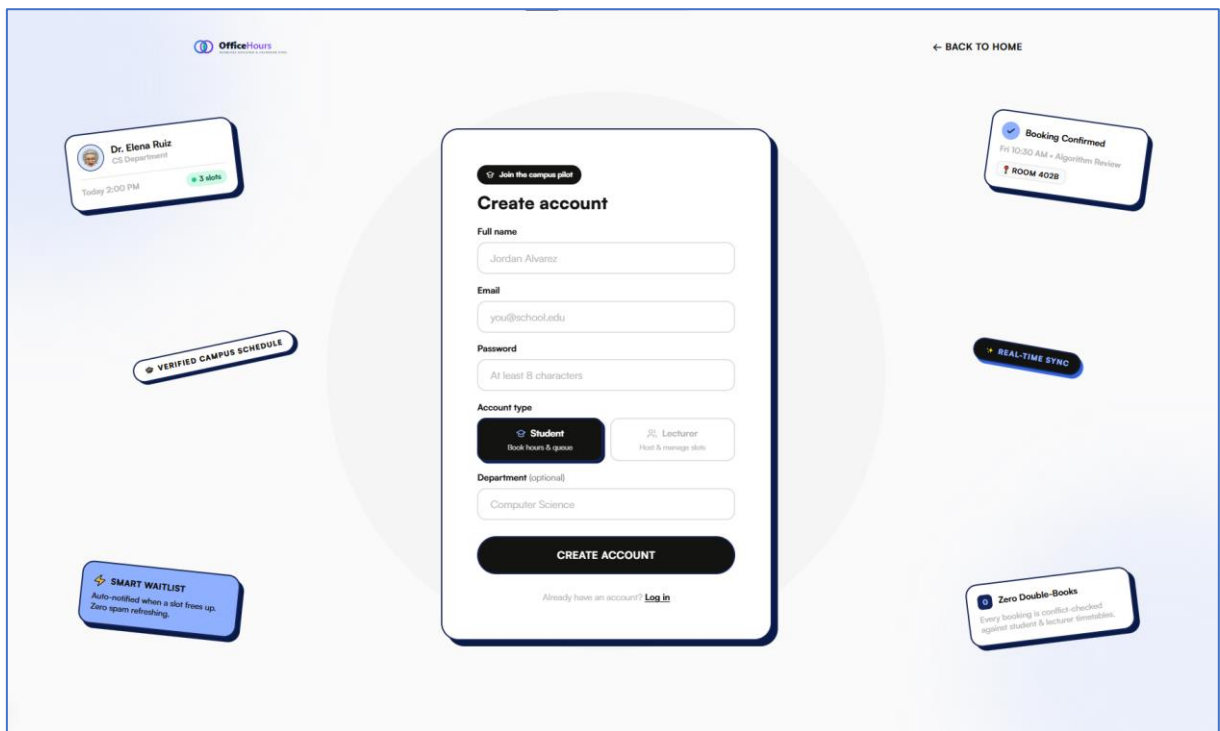


Figure 16. Glassmorphic Register Screens

- Password Retrieval Process: The recovery page provides users with an option to steal a password using their emails while remaining informed about the progress of their entry.

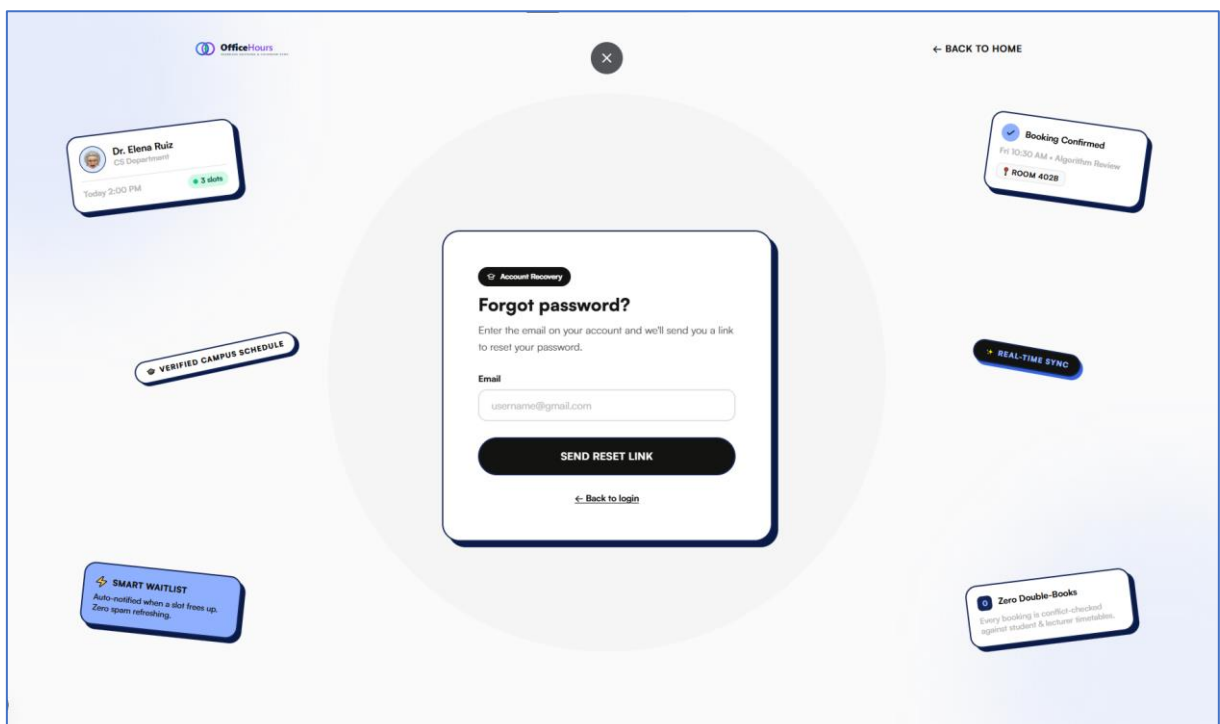


Figure 17. Password Recovery Flow

Shared Authenticated Workspaces:

- Role-Based Dashboard Homepage: In a short duration of logging in, it changes to a straightforward interface devoid of distractions. Provides tailored displays for Students

(future bookings), Lecturers (outstanding approvals), and Administrators (system metrics).

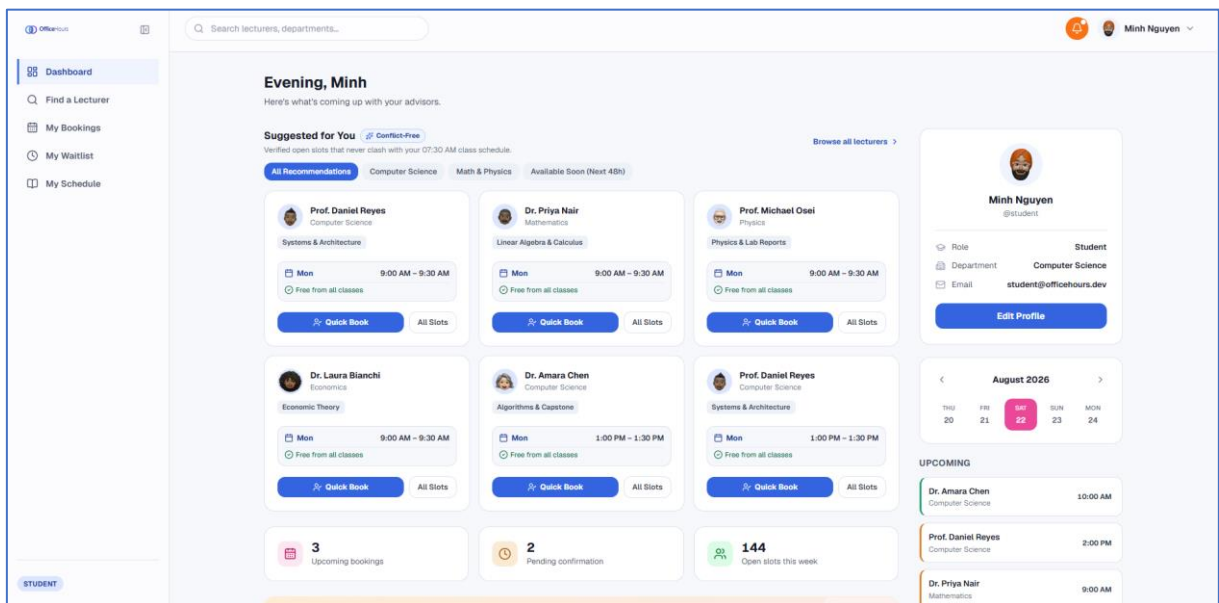


Figure 18. Role-Aware Dashboard Home

- My Profile & Security: Lets users change personal information and modify notification settings, depending on Terms and Conditions.

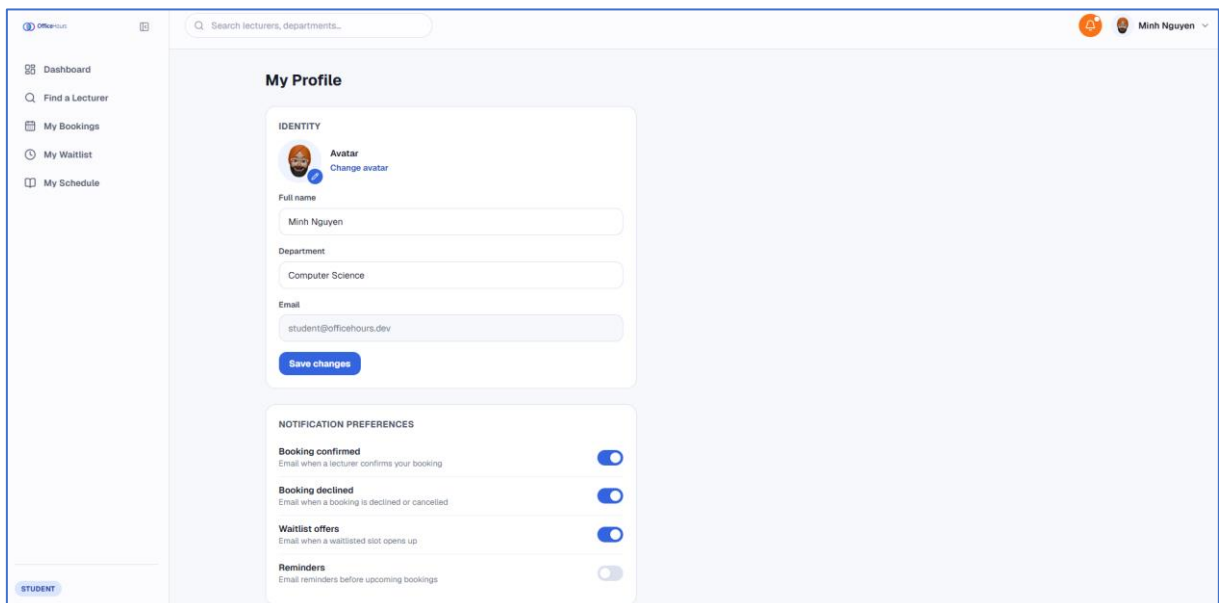


Figure 19. My Profile & Security

- Notification Center: Sorting alerts by tabulation into sections (any unread/any read), it makes the client's system ready for simultaneous notification processing.

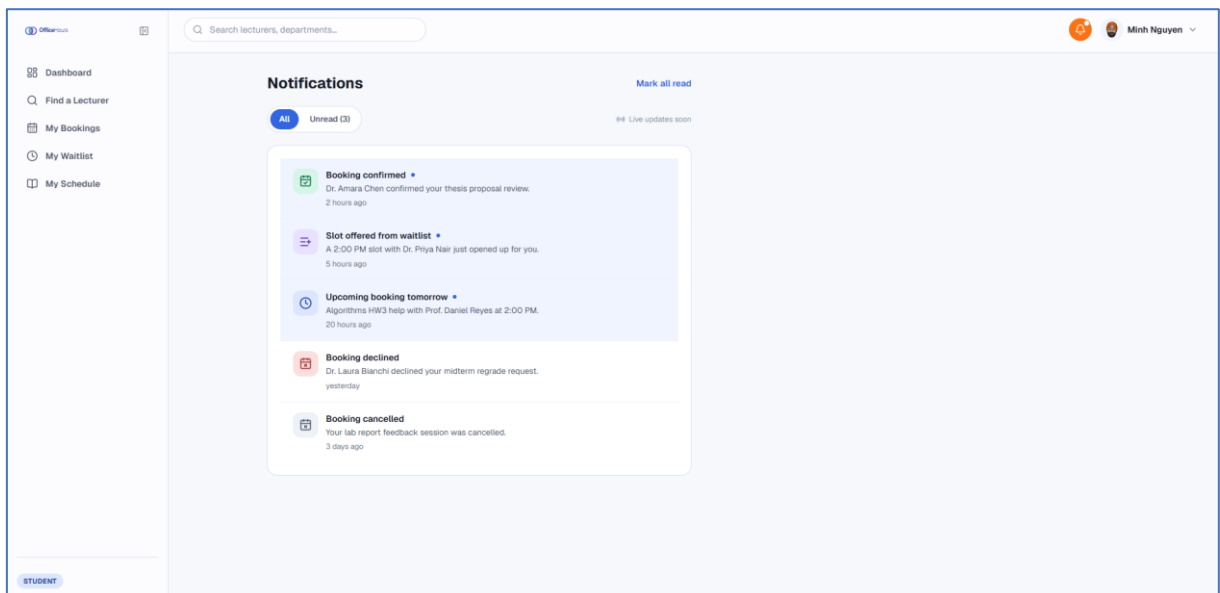


Figure 20. Notification Center

- My Bookings & Timeline View: In its design has a data table sorted in two ways and timer-charts plus an additional timeline, showing the process since the initial booking up to its completion.

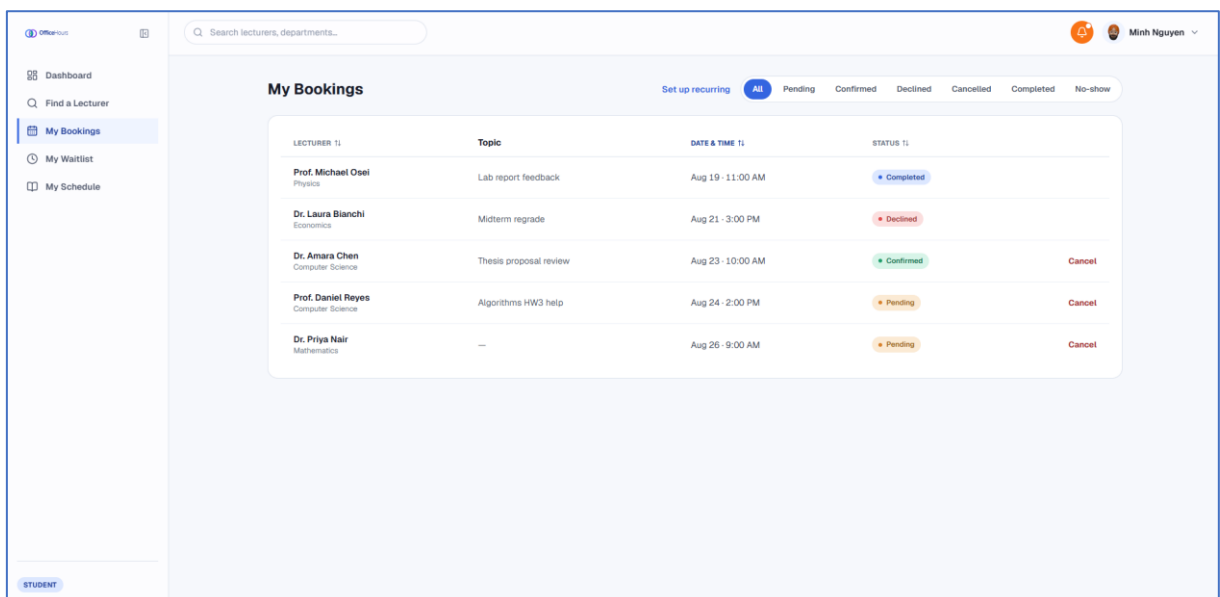


Figure 21. My Bookings & Timeline View

Student Operations Views:

- Lecturer Search & Directory: There's a search-enabled Directory, so one can search for people depending on their name or position.

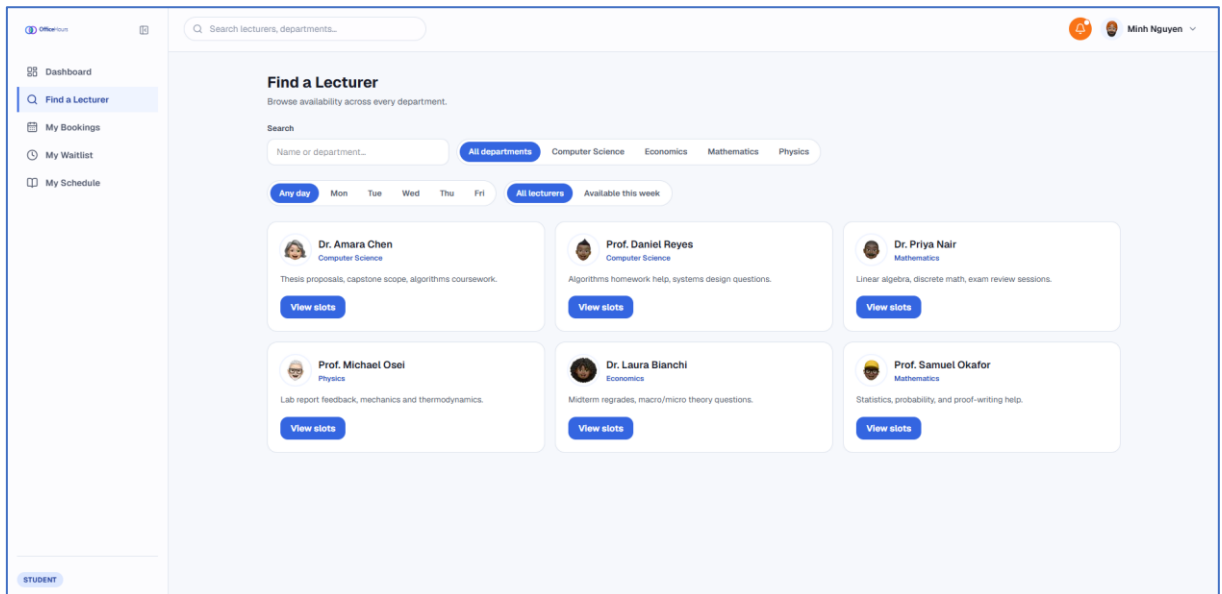


Figure 22. Lecturer Search & Directory

- My Class Schedule: The schedule of classes customized by the user shows in the view as a timetable and agenda.

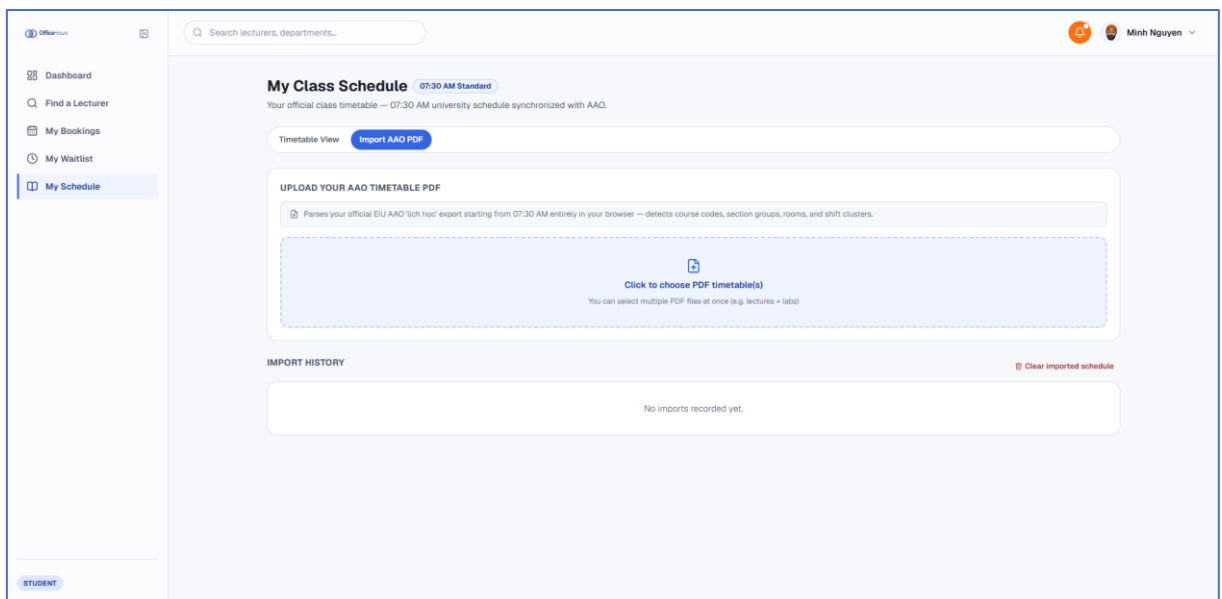


Figure 23. Import AAO PDF Dashboard

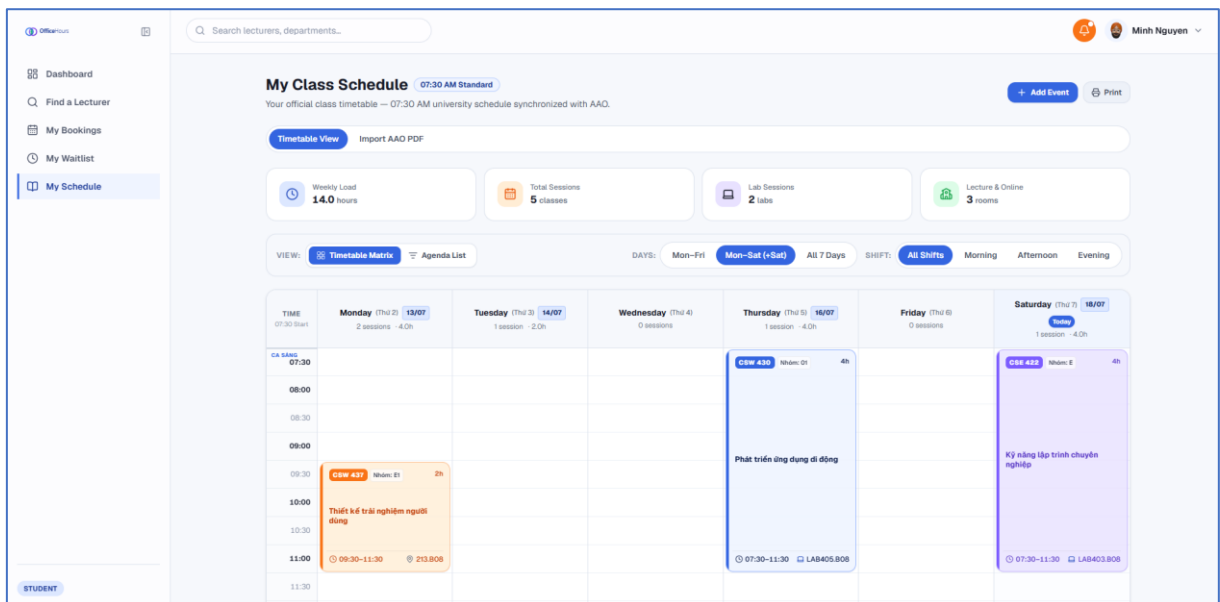


Figure 24. Time Table View

- Weekly Slot Picker: Shows scheduled appointments on a weekly table view, excluding slots conflicting with classes.

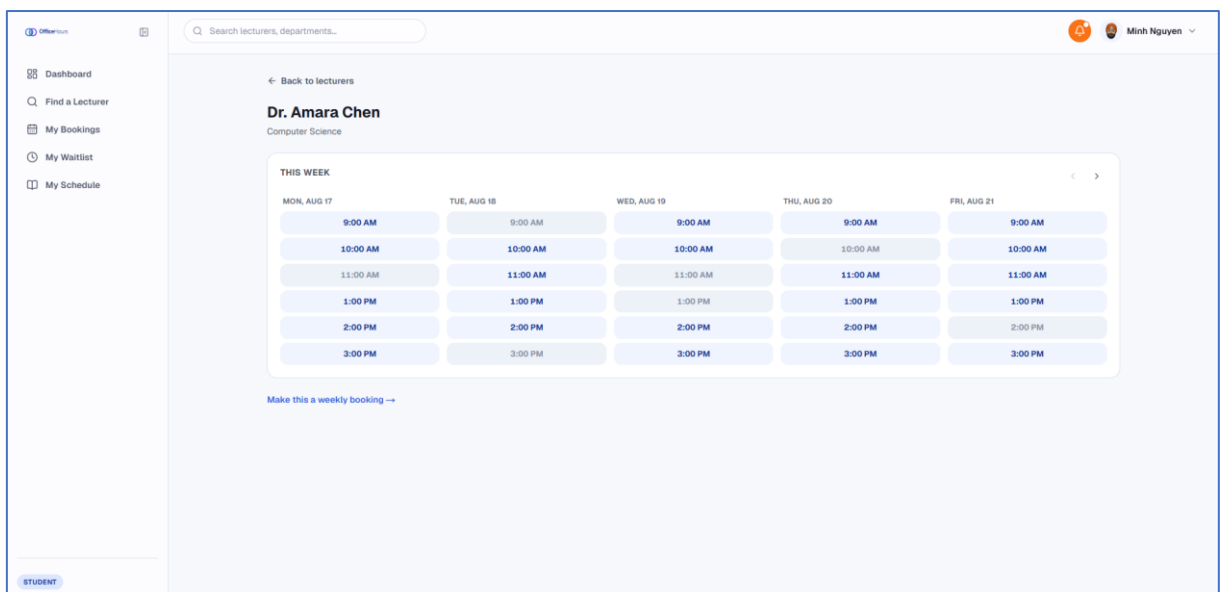


Figure 25. Weekly Slot Picker

- The Group Booking and Recurring configuration allows for imperative multi-party email invites through a ready-made structure using the software as well as being able to automatically come up with appointment dates all through the semester on its own.

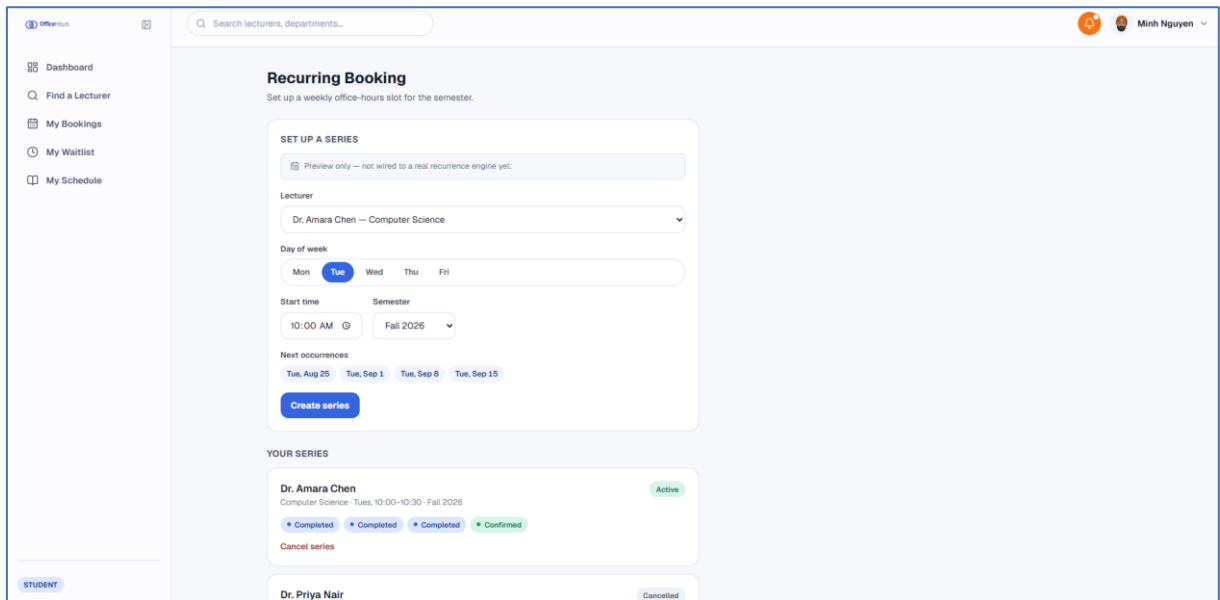


Figure 26. Group Booking & Recurring Setup

- The Waitlist Management Dashboard has a display of waiting list positions and a timer countdown for newly available spots.

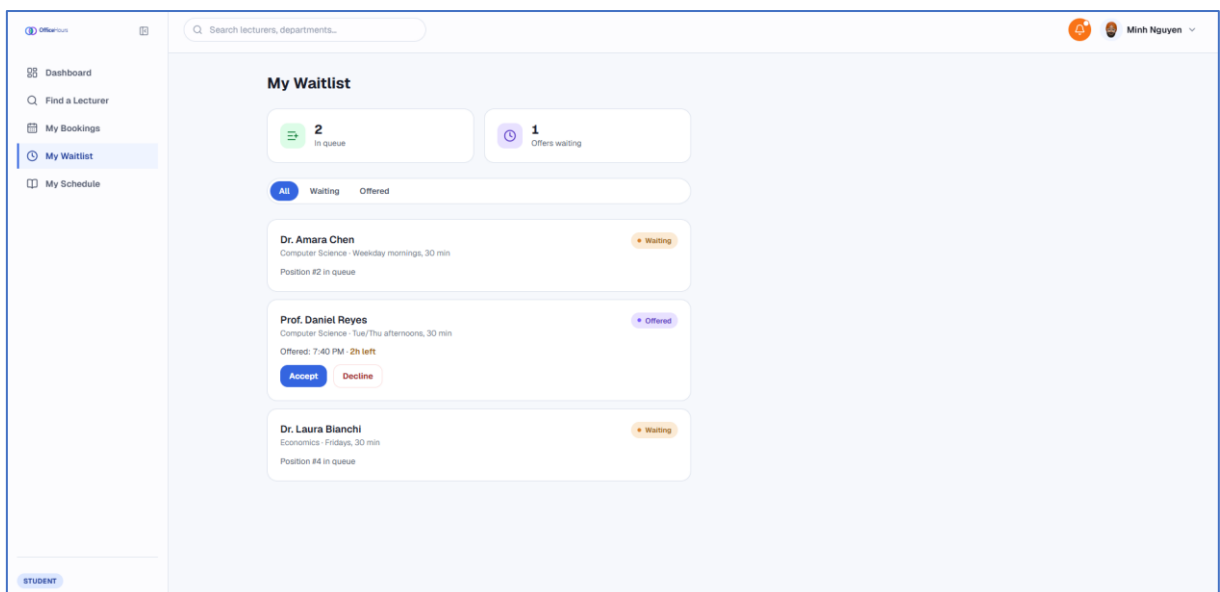


Figure 27. Waitlist Management Dashboard

Lecturer Operations Views:

- The Availability Rule Management editor allows defining of regular consultation hours every week with the exemption of special days.

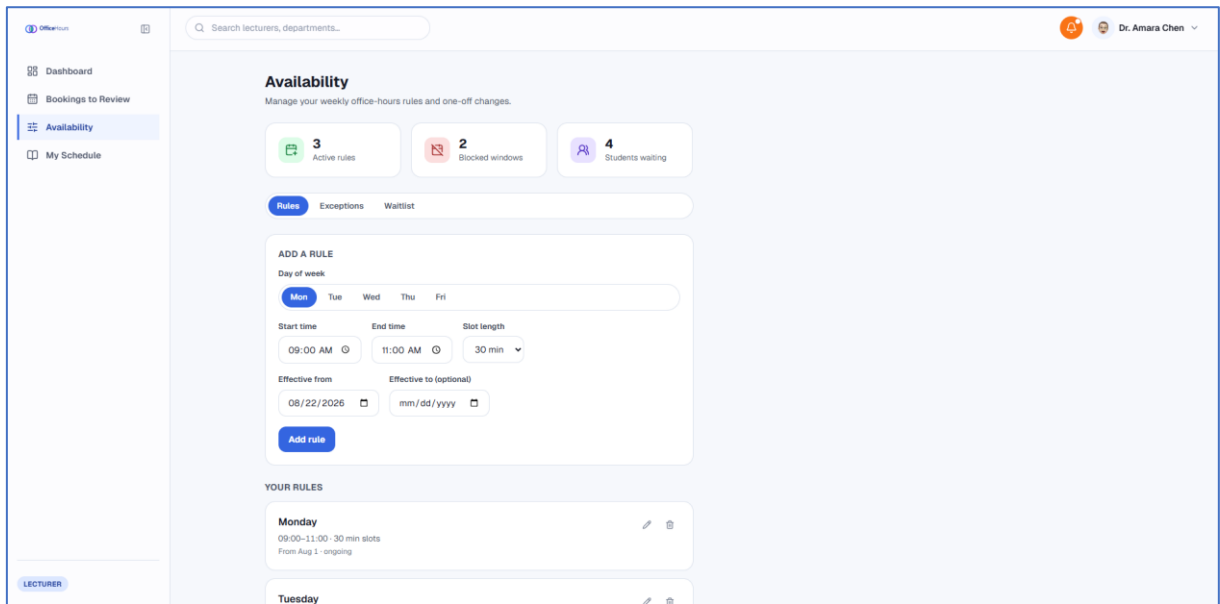


Figure 28. Availability Rules Manager

- The Slot Waitlist Queue shows the position along with the countdown timer for every consultation spot available for students.

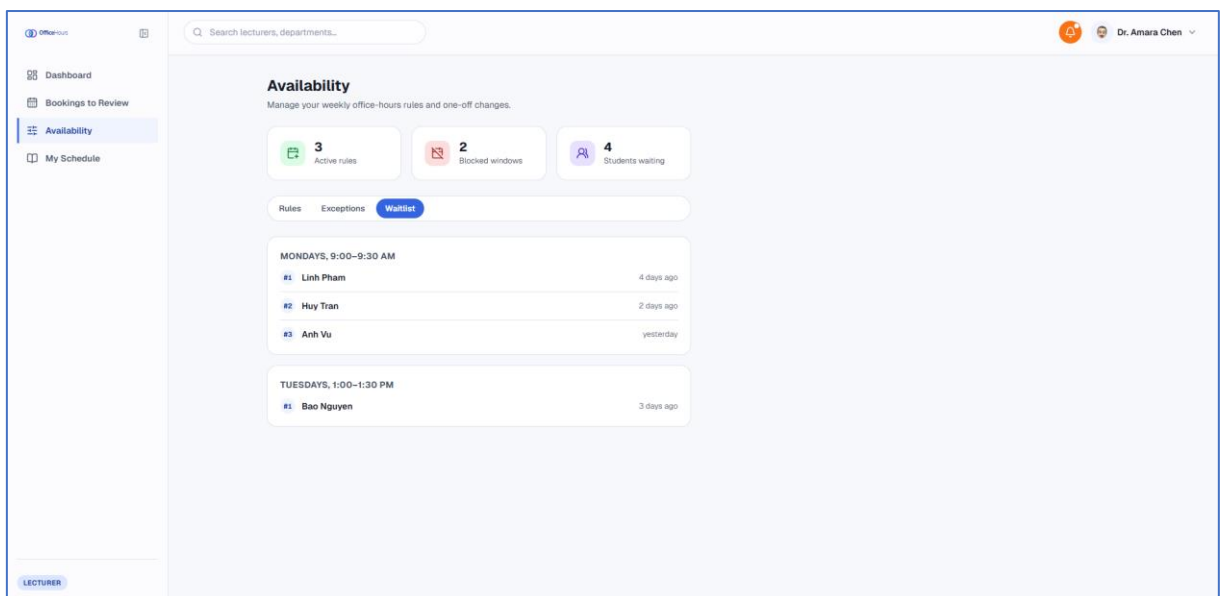


Figure 29. Slot Waitlist Queue

- My Teaching Schedule is a custom page created specifically for the professors to manage their teaching schedules with the ability to import their teaching timetable PDFs or insert/delete blocks manually.

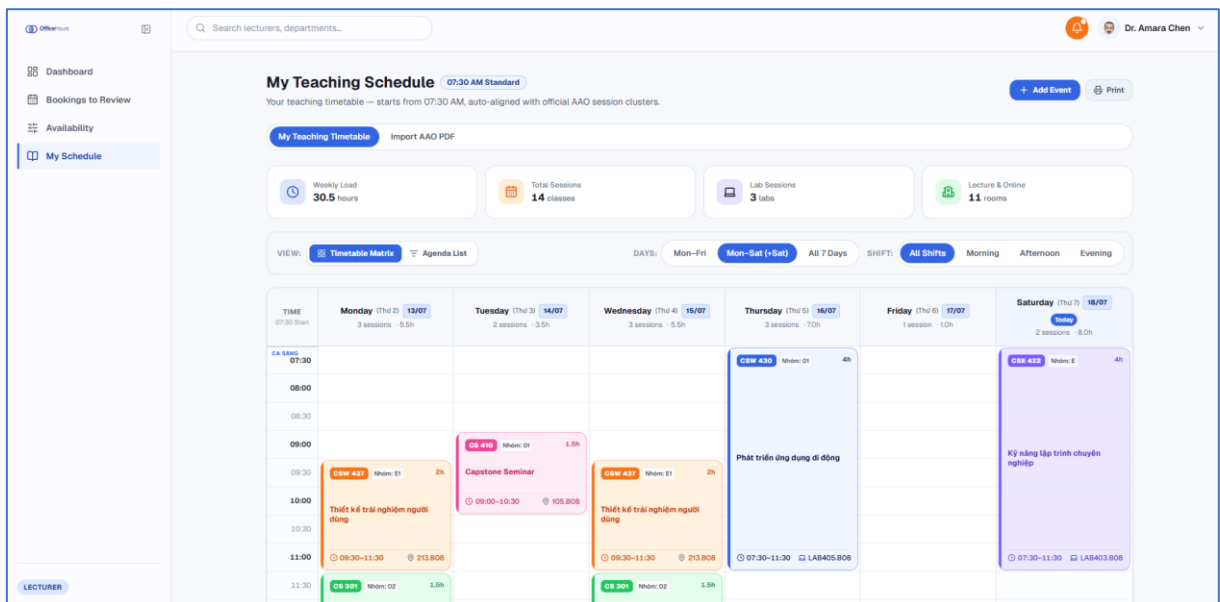


Figure 30. My Teaching Schedule

- Booking Review & Attendance Journal does mass approving and rejecting of the booking requests and making notes after the consultations.

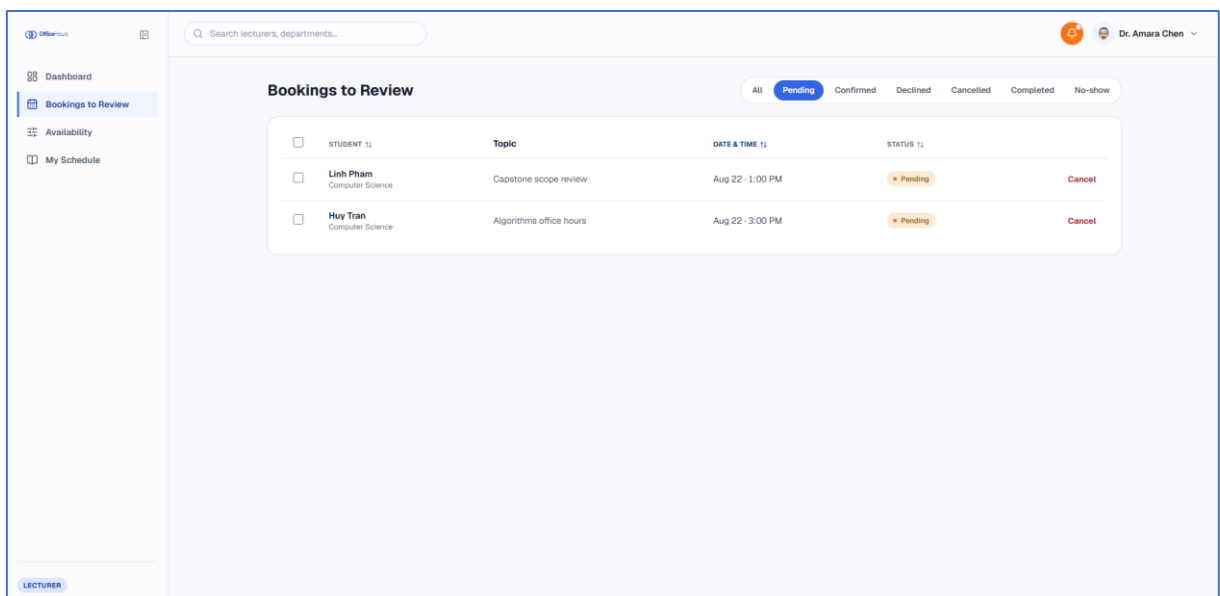


Figure 31. Booking Review & Attendance Journal

Administrative & Research Analytics Views:

- The User and Semester Management allows for updates within the structures and boundaries of the departments, as well as the semester boundaries.

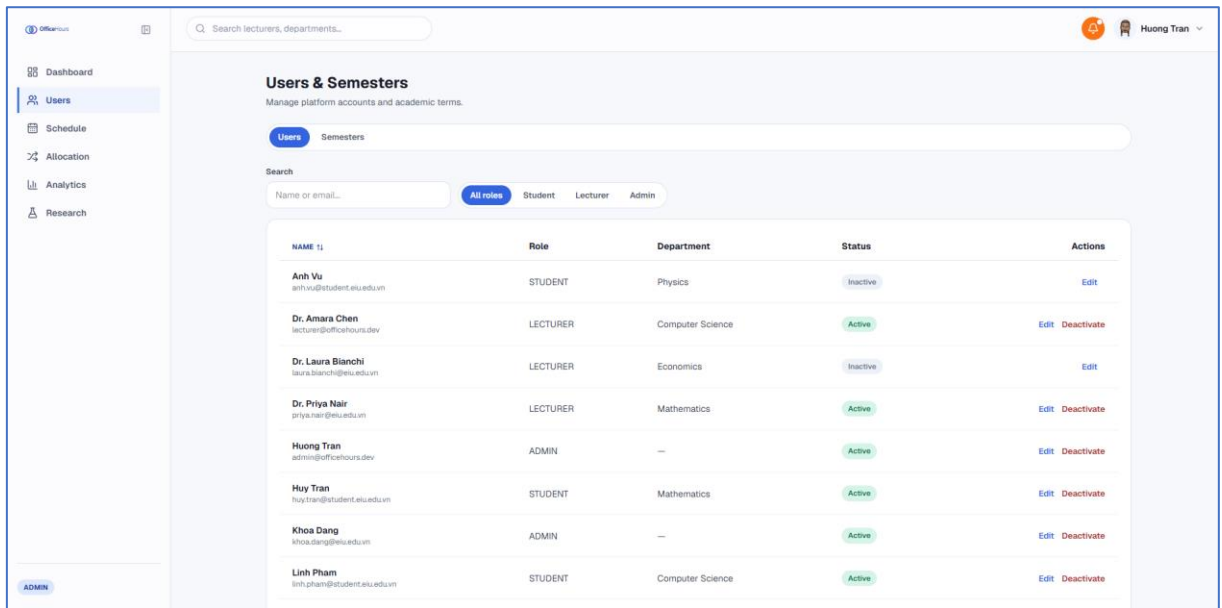


Figure 32. User & Semester Management

- PDF Timetable Importer: The same pdf.js-based tool that students and lecturers use for their timetable is now available for the administrative purposes of assigning schedules to users, where the owner picker will determine who is the subject of the educational process, after which the pdf file is uploaded or manual entry of busy periods will be made to the schedule of the selected users' timetable, and slot search function will help to find free slots in the schedules of all lecturers.

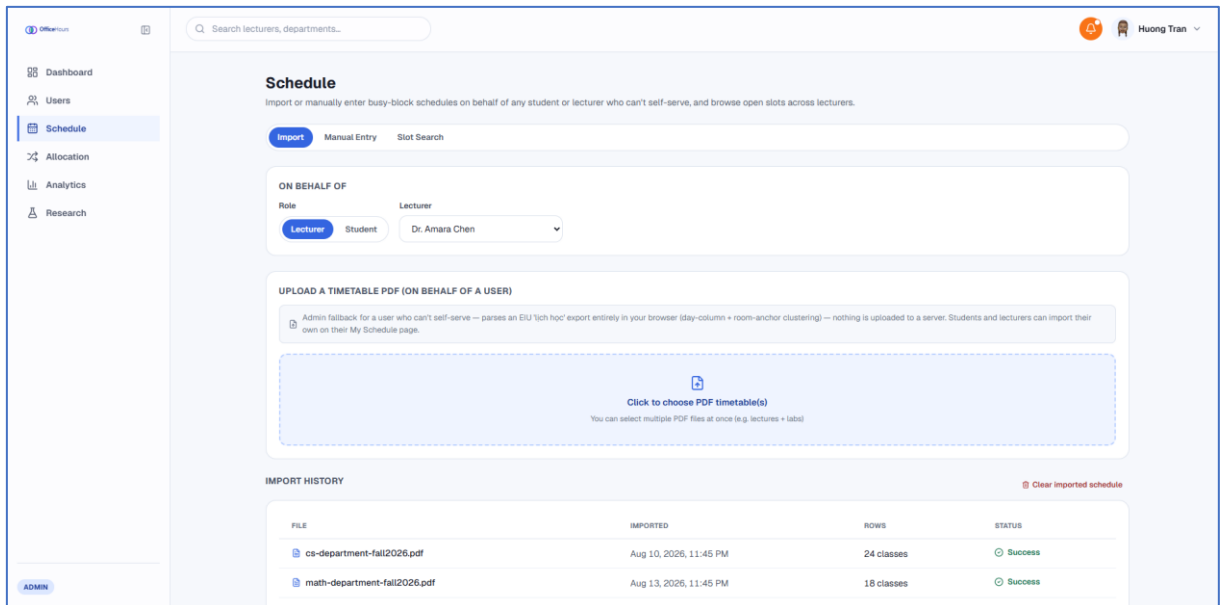


Figure 33. PDF Timetable Importer

- Allocation Policy Manager & Audit Log: The module allows the setting up of various waitlist policies such as FCFS, NEED and HYBRID and delivering an auditing report with capability of changing it manually.

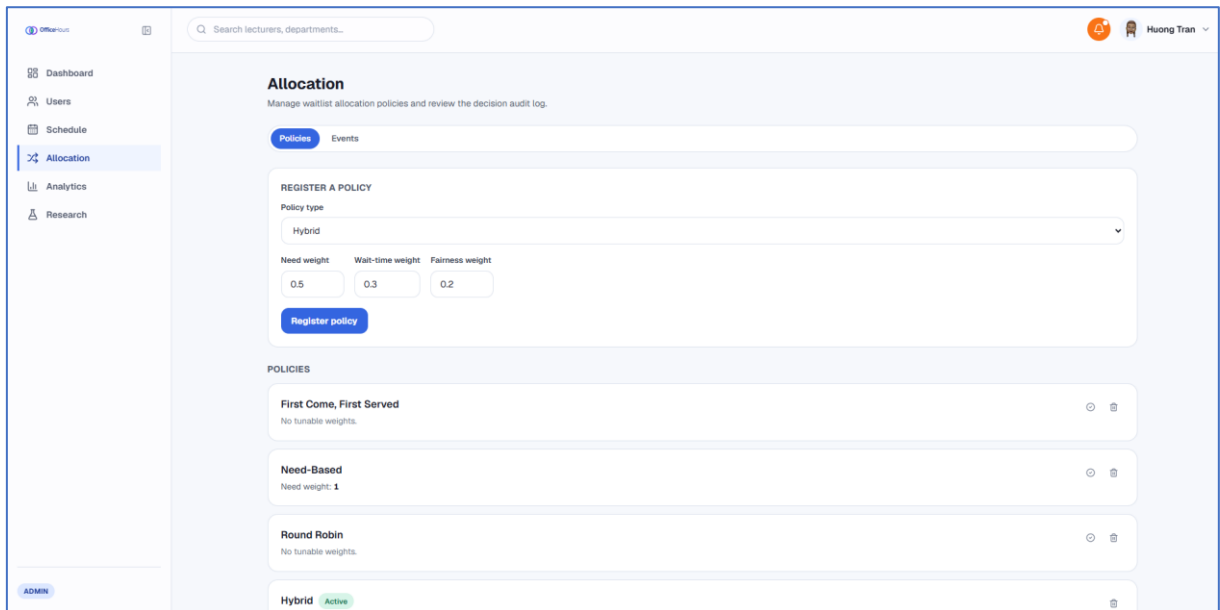


Figure 34. Allocation Policy Manager & Audit Log

- Analytics & Policy Benchmark Runner: The module presents a visual representation of workload of advisors, Lorenz curves, Gini indexes and performance vs equity charts with the help of Recharts.

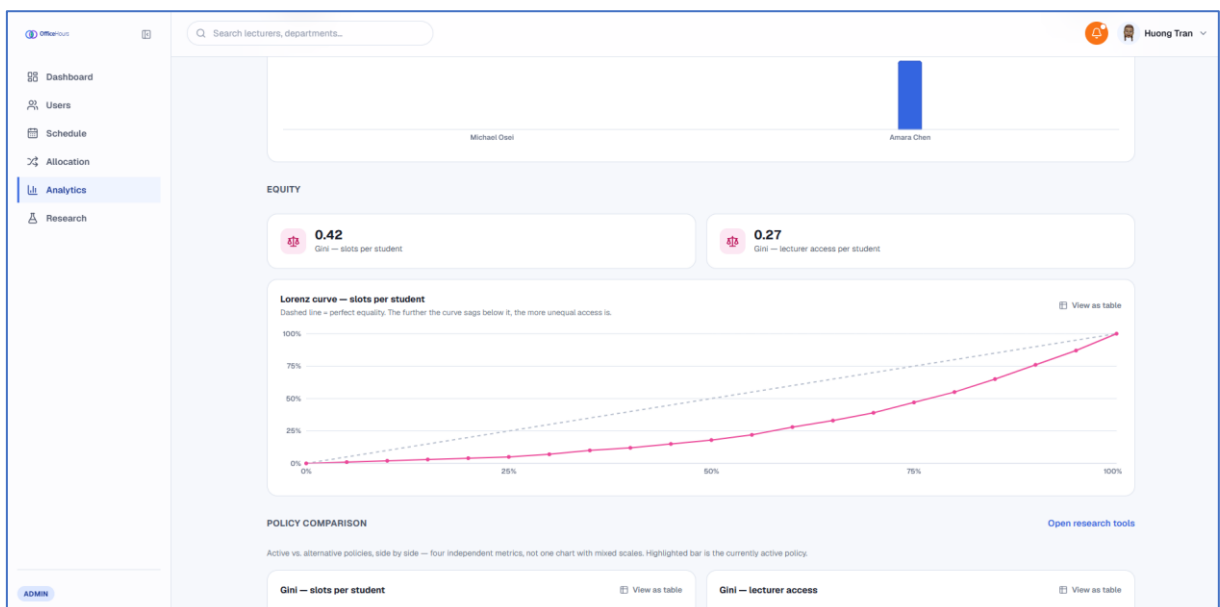


Figure 35. Analytics & Policy Benchmark Runner

4.2.2. Frontend Evaluation and Discussion

Evaluation of the web client was devoted to such aspects of its performance as the usability of the interface, client-side data handling and layout performance in different viewport sizes.

Layout Responsiveness and Render Performance:

- The removal of graphic-heavy CSS decorators (like backdrop blur filters) from the authentication screens eliminated layout glitches while displaying the data-dense

dashboards. The data tables which relied on the tabular-nums CSS property managed to maintain typographic positions within the tables being displayed.

Client-Side Schedule Conflict Prevention:

- The client-side schedule assessments were performed through the timetable conflict checking tool, allowing conflicts to be verified before users submitted the application form. Those conflicts were automatically detected during the rendering of browsers and users were given immediate feedback in terms of disabled consultations so they would know that they tried to choose an invalid time.

Algorithmic Visualization and Fairness Metrics:

- The analytics module proved to be successful in terms of performing client-side calculations driven by the Gini coefficient formula:

$$G = \frac{\sum_{i=1}^n \sum_{j=1}^n |x_i - x_j|}{2n^2 \bar{x}}$$

- Through the use of simulated demands, the front-end dashboard managed to produce the curves showing the differences between the efficiency of using First-Come-First-Served and Hybrid methods of distribution, allowing to conclude that the user interface is able to do highly complicated computations and visualization.

CHAPTER 5. CONCLUSION AND FUTURE WORKS

5.1. Conclusion

The frontend web application development for the academic consulting and office hours management platform gave us great real-world experience in both modern web engineering and user interface design. The project led the learning process through the software engineering lifecycle, beginning with the conception and the requirements analysis and finishing with interface design, implementation of components, and client-side usability testing. It represented a chance to apply theoretical knowledge of resources' allocation in the implementation of the responsive web application while mastering essential tools for developers, such as Git as a version-control tool, frameworks Next.js/React, and Tailwind CSS for modular design of user interface components.

Working on this platform provided good knowledge in designing user-centered software for academic purposes. First of all, it was designed the platform in such a way that made scheduling consultations and tracking teachers' availability easier. Components such as role-oriented dashboards, weekly time-slots choice, and instant accessibility monitoring through Gini coefficients and Lorenz curves were implemented on this website.

The execution of the project proved useful beyond the technical implementation since it contributed to the development of the skills that can be classified as problem-solving and critical thinking. The development process involved dealing with a number of issues related to project implementation such as the response to complex client state changes, interpreting timetable documentation with the help of PDF.js, and developing visual representations of the pros and cons of the algorithm using Recharts. Moreover, the task of adhering to specific deadlines, keeping clean architecture of the code, and writing down the decisions made during the design process helped me develop my self-studying and organized engineering skills enabling me for future software development in the best possible way.

5.2. Future Works

The current application has developed a full frontend setup that provides basic features such as appointment scheduling based on user roles, waitlist queue management, and visualization of allocation policies. In order to advance it from an original state on the client side to a fully working academic ecosystem for an enterprise, a number of important changes need to be made in future releases.

5.2.1. Backend API & Persistence Layer Implementation

The first step on the way of development will be creating a reliable backend. The creation of RESTful APIs based on ASP.NET Core or Node.js and a relational database system –

PostgreSQL or SQL Server – will replace the mock data handlers from the client side (getMockOfficeHours). This layer will ensure secure JWT authentication, handle data persistence, and permit reasonable storage restrictions to avoid double-booking at the level of the database.

5.2.2. Mobile Application Development

To enable users to access the application when on the move, it will be necessary to create mobile applications based on React Native or Flutter. Mobile applications will provide users with push notifications for confirming bookings, checking the schedule for future activities in the mobile calendar, and scanning QR codes.

5.2.3. Student Information System (SIS) API Integration

By connecting the educational system directly with the institutional SIS as well as LMS, schedule synchronization will be completely automated as the API integration will be done live. This bi-directional integration will replace manual importing of timetables in .pdf or .csv formats. Furthermore, it will enable system detection of all schedule changes and automatic modification of the system without participation of the administration.

5.2.4. Real-time Communication & Notification Engine

With the use of WebSockets technology or Server-Sent Events technology, the whole notification system will have its communication mechanism upgraded and made totally real-time. Students on the waiting list will get notified about the slots available right away.

5.2.5. Predictive Machine Learning for Demand Forecasting

Introducing predictive machine learning models into the system will improve administrative decision-making. It analyzes past booking patterns and predicts an influx of consultations during midterm and final exam times, making it possible for professors and administrators to manage office hours ahead of time before any scheduling conflicts happen. The planned improvements will ultimately turn the system into an intelligent, integrated, highly scalable academic scheduling system able to serve complex institutional processes.

REFERENCES

- [1] Vercel, "Next.js Documentation", Next.js, 2026. [Online]. Available: <https://nextjs.org/docs>. [Accessed: Aug, 18, 2026]
- [2] Meta Platforms, Inc., "React Documentation," *React*, 2026. [Online]. Available: <https://react.dev>. [Accessed: Aug, 18, 2026].
- [3] Tailwind Labs Inc., "Tailwind CSS Documentation," *Tailwind CSS*, 2026. [Online]. Available: <https://tailwindcss.com/docs>. [Accessed: Aug, 18, 2026].
- [4] Framer B.V., "Framer Motion Documentation," *Framer*, 2026. [Online]. Available: <https://motion.dev>. [Accessed: Aug, 18, 2026].
- [5] Lucide Contributors, "Lucide Icons Documentation," *Lucide*, 2026. [Online]. Available: <https://lucide.dev>. [Accessed: Aug, 18, 2026].
- [6] Recharts Community, "Recharts Documentation," *Recharts*, 2026. [Online]. Available: <https://recharts.org>. [Accessed: Aug, 18, 2026].
- [7] P. Srinivas, "Rough.js Documentation," *Rough.js*, 2026. [Online]. Available: <https://roughjs.com>. [Accessed: Aug, 18, 2026].
- [8] Broadcom Inc., "Spring Boot Reference Documentation," *Spring*, 2026. [Online]. Available: <https://spring.io/projects/spring-boot>. [Accessed: Aug, 18, 2026].
- [9] PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," *PostgreSQL*, 2026. [Online]. Available: <https://www.postgresql.org/docs>. [Accessed: Aug, 18, 2026].
- [10] GitHub, Inc., "GitHub Docs," *GitHub*, 2026. [Online]. Available: <https://docs.github.com>. [Accessed: Aug, 18, 2026].